Faculty of Engineering

Department of Software Engineering and Intelligent Systems

كلية هندسة الذكاء الصنعي

قسم هندسة البرمجيات والنظم الذكية

نظام إدارة التعلم LMS

Learning Management System LMS

Senior-2 Project- Completed the requirements for obtaining a bachelor's degree in Software Engineering and Intelligent Systems

Prepared By:

Sulaf Walid Al-Samara

Supervised By:

Dr. Akram Masouh

October 2026

SUPERVISOR CERTIFICATION

I Certify that the Preparation of this Project entitled by ……
, prepared by ……..…
……
……., was made under my supervision at Department of Informatics Engineering in partial fulfillment of the Requirements for the Degree of Bachelors of Software Engineering and Information Systems

Name: …… Signature: …..…….. Date:

Abstract

The rapid digitization of education highlights a critical problem: local institutions struggle to adopt global Learning Management Systems (LMS) due to linguistic barriers, complex localization efforts, and heavy cloud dependencies. This research is justified by the urgent need for a culturally aligned educational ecosystem that supports native Arabic interfaces and local pedagogical workflows without relying on expensive foreign services. The primary objective is to design, develop, and evaluate a fully functional, Arabic-native LMS that streamlines course delivery, assessment management, and academic administration. The proposed solution is a web-based platform utilizing a Model-View-Template (MVT) architecture. The backend is engineered using the Django framework (Python) with an SQLite database for structured data persistence, while the frontend employs HTML5, CSS3, and Bootstrap 5 to ensure a responsive, Right-to-Left (RTL) user experience. The expected results include a stable, maintainable, and highly efficient LMS prototype that eliminates localization overhead, provides robust role-based management, and facilitates seamless educational processes tailored to regional academic standards.

Keywords: Learning Management System, Django Framework, Arabic Localization, MVT Architecture, Educational Technology, RTL Web Design.

الملخص

يبرز الترقمي السريع للتعليم مشكلة جوهرية تتمثل في صعوبة المؤسسات المحلية في تبني أنظمة إدارة التعلم (LMS) العالمية بسبب الحواجز اللغوية، وتعقيدات التوطين، والاعتماد الكبير على الخدمات السحابية. يُبرر هذا البحث الحاجة الملحة لبيئة تعليمية متوافقة ثقافياً تدعم الواجهات العربية الأصلية وتلبي سير العمل البيداغوجي المحلي دون الاعتماد على خدمات أجنبية مكلفة. يهدف هذا المشروع بشكل رئيسي إلى تصميم وتطوير وتقييم نظام إدارة تعلم متكامل ومتمحور حول اللغة العربية لتبسيط تسليم المقررات، وإدارة التقييمات، والإدارة الأكاديمية. الحل المقترح هو منصة ويب تعتمد معمارية (MVT)، حيث يُبنى الجانب الخلفي على إطار عمل Django بلغة Python مع قاعدة بيانات SQLiteلتخزين البيانات، بينما تستخدم الواجهة الأمامية HTML5 و CSS3وBootstrap 5 لضمان تجربة مستخدم متجاوبة وتدعم الاتجاه من اليمين إلى اليسار .(RTL) تشمل النتائج المتوقعة نموذجاً أولياً مستقراً وسهل الصيانة وعالي الكفاءة يلغي أعباء التوطين، ويوفر إدارة قوية للأدوار، ويسهل العمليات التعليمية المتوافقة مع المعايير الأكاديمية الإقليمية.

الكلمات المفتاحية :نظام إدارة التعلم، إطار عمل جانغو، التوطين العربي، معماريةMVT ، تكنولوجيا التعليم، تصميم الويب الداعم لـ.RTL

List of Abbreviations and Symbols

Abbreviation	English Meaning	Arabic Meaning
LMS	Learning Management System	نظام إدارة التعلم
MVT	Model-View-Template	معمارية النموذج–العرض–القالب
RTL	Right-to-Left	من اليمين إلى اليسار
ORM	Object-Relational Mapping	تعيين الكائنات والعلاقات
RBAC	Role-Based Access Control	التحكم بالوصول المبني على الأدوار
RTM	Requirements Traceability Matrix	مصفوفة تتبع المتطلبات
UI/UX	User Interface / User Experience	واجهة المستخدم / تجربة المستخدم
API	Application Programming Interface	واجهة برمجة التطبيقات
DBMS	Database Management System	نظام إدارة قواعد البيانات

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION 1

1.1 PROJECT BACKGROUND: 2
1.2 PROBLEM DEFINITION: 2
1.3 PROJECT IMPORTANCE: 3
1.4 PROJECT OBJECTIVES: 3
1.5 PROJECT SCOPE: 4
1.6 REPORT CONTRIBUTIONS: 5
1.7 REPORT ORGANIZATION: 6

CHAPTER 2 BASIC CONCEPTS AND REFERENCE STUDIES 8

2.1 BASIC CONCEPTS: 9
2.2 REFERENCE STUDIES AND PREVIOUS WORK: 10
2.3 COMPARISON OF PREVIOUS WORK: 12
2.4 RESEARCH GAP: 14

CHAPTER 3 PROJECT METHODOLOGY AND REQUIREMENT ANALYSIS 16

3.1 SELECTED DEVELOPMENT METHODOLOGY: 17
3.2 GENERAL FUNCTION REQUIREMENTS: 17
3.3 NON-FUNCTIONAL REQUIREMENTS: 18
3.4 PROJECT SPECIFIC FUNCTIONAL REQUIREMENTS: 19
3.5 FEASIBILITY AND RISKS: 24
3.5.1 Feasibility Study 24
3.5.2 Risk Management 25
3.6 PROJECT PLAN 26

CHAPTER 4 ANALYTICAL AND DESIGN STUDY 28

4.1 SYSTEM ARCHITECTURE: 29
4.2 USE CASE DIAGRAM 30
4.2.1 Core Use Case Specifications: 34
4.3 ACTIVITY DIAGRAM: 39
4.4 SEQUENCE DIAGRAM: 44
4.5 CLASS DIAGRAM: 47
4.6 COMPONENT DIAGRAM: 47
4.7 ENTITY-RELATIONSHIP DIAGRAM (ERD): 49
4.8 SITEMAP: 63
4.9 INITIAL TEST CASES (UNIT TESTS): 64
4.10 AI AND ALGORITHMS DESIGN: 64
4.11 REQUIREMENTS TRACEABILITY MATRIX (RTM): 65

CHAPTER 5 IMPLEMENTATION 67

5.1 IMPLEMENTATION ENVIRONMENT: 68
5.2 PROJECT SOFTWARE STRUCTURE: ..... **ERROR! BOOKMARK NOT DEFINED.**
5.3 MODULES IMPLEMENTATION: ..... **ERROR! BOOKMARK NOT DEFINED.**
5.4 INTEGRATION AND PROGRAMMING INTERFACES: ..... **ERROR! BOOKMARK NOT DEFINED.**
5.5 UPDATED REQUIREMENTS TRACEABILITY MATRIX (RTM): ..... **ERROR! BOOKMARK NOT DEFINED.**
5.6 DEPLOYMENT AND OPERATION: ..... **ERROR! BOOKMARK NOT DEFINED.**

CHAPTER 6 TESTING AND EVALUATION ..... ERROR! BOOKMARK NOT DEFINED.

6.1 INTEGRATION AND SYSTEM TESTING STRATEGY AND PLAN: ..... **ERROR! BOOKMARK NOT DEFINED.**
6.2 TEST CASES AND RESULTS: ..... **ERROR! BOOKMARK NOT DEFINED.**
6.3 PERFORMANCE EVALUATION: ..... **ERROR! BOOKMARK NOT DEFINED.**
6.4 AI/HEURISTIC MODEL EVALUATION: ..... **ERROR! BOOKMARK NOT DEFINED.**
6.5 RESULTS ANALYSIS: ..... **ERROR! BOOKMARK NOT DEFINED.**

CHAPTER 7 CONCLUSION AND FUTURE WORK ERROR! BOOKMARK NOT DEFINED.

7.1 CONCLUSION: ..... **ERROR! BOOKMARK NOT DEFINED.**
7.2 FUTURE WORK: ..... **ERROR! BOOKMARK NOT DEFINED.**
REFERENCES: ..... **ERROR! BOOKMARK NOT DEFINED.**

APPENDIX A FULL REQUIREMENTS TRACEABILITY MATRIX ERROR! BOOKMARK NOT DEFINED.

APPENDIX B FULL SYSTEM TEST CASES AND RESULTS... ERROR! BOOKMARK NOT DEFINED.

APPENDIX C SYSTEM UI SCREENSHOT ..... ERROR! BOOKMARK NOT DEFINED.

LIST OF TABLES

TABLE (2.1) COMPARATIVE ANALYSIS OF PREVIOUS SYSTEMS AND STUDIES ... 12
TABLE (3.1) NON-FUNCTIONAL REQUIREMENTS MATRIX 18
TABLE (3.2) REQUIREMENTS DATABASE (RDB) 20
TABLE (3.3) RISK REGISTER 25
TABLE (4.1) USER AUTHENTICATION (SIGN IN) 34
TABLE (4.2) CREATE AND MANAGE COURSE 35
TABLE (4.3) ASSIGNMENT CREATION AND GRADING WORKFLOW 36
TABLE (4.4) QUIZ EXECUTION AND AUTOMATED OBJECTIVE GRADING 37
TABLE (4.5) IDENTIFY AT-RISK STUDENTS (ACADEMIC ANALYTICS) 38
TABLE (4.6) RELATIONAL SCHEMA ADDITIONS 50
TABLE (4.7) INITIAL UNIT TEST CASES 64
TABLE (4.8) REQUIREMENTS TRACEABILITY MATRIX (RTM) 66
TABLE (5.1) IMPLEMENTATION TRACEABILITY FOR ADVANCED MODULES ..... ERROR! BOOKMARK NOT DEFINED.
TABLE (6.1) SYSTEM TEST CASES AND RESULTS (SELECTED ADVANCED MODULES) ..... ERROR! BOOKMARK NOT DEFINED.
TABLE (6.2) FINAL REQUIREMENTS TRACEABILITY MATRIX (RTM) - SUMMARY ..... ERROR! BOOKMARK NOT DEFINED.

LIST OF FIGURES

FIGURE (3.1) GANTT CHART **26**

FIGURE (4.1) HIGH-LEVEL SYSTEM ARCHITECTURE BLOCK DIAGRAM **30**

FIGURE (4.2) GENERAL USE CASE DIAGRAM **31**

FIGURE (4.3) USE CASE DIAGRAM FOR PROJECT 1 ..... **32**

FIGURE (4.4) USE CASE DIAGRAM FOR CURRENT PROJECT **33**

FIGURE (4.5) ACTIVITY DIAGRAM FOR USER AUTHENTICATION (SIGN IN) **39**

FIGURE (4.6) ACTIVITY DIAGRAM FOR CREATE AND MANAGE COURSE **40**

FIGURE (4.7) ACTIVITY DIAGRAM ASSIGNMENT CREATION AND GRADING WORKFLOW **41**

FIGURE (4.8) ACTIVITY DIAGRAM ASSIGNMENT CREATION AND GRADING WORKFLOW **42**

FIGURE (4.9) ACTIVITY DIAGRAM FOR IDENTIFY AT-RISK STUDENTS (ACADEMIC ANALYTICS) **43**

FIGURE (4.10) SEQUENCE DIAGRAM FOR USER AUTHENTICATION (SIGN IN) **44**

FIGURE (4.11) SEQUENCE DIAGRAM FOR CREATE AND MANAGE COURSE **45**

FIGURE (4.12) SEQUENCE DIAGRAM ASSIGNMENT CREATION AND GRADING WORKFLOW **45**

FIGURE (4.13) SEQUENCE DIAGRAM ASSIGNMENT CREATION AND GRADING WORKFLOW **46**

FIGURE (4.14) SEQUENCE DIAGRAM FOR IDENTIFY AT-RISK STUDENTS (ACADEMIC ANALYTICS) **46**

FIGURE (4.15) SYSTEM CLASS DIAGRAM **47**

FIGURE (4.16) COMPONENT DIAGRAM **48**

FIGURE (4.17) ENTITY-RELATIONSHIP DIAGRAM (ERD) **49**

FIGURE (4.18) SYSTEM SITEMAP **63**

FIGURE (C.1) LANDING PAGE ..... ERROR! BOOKMARK NOT DEFINED.

FIGURE (C.2) ADMIN DASHBOARD ..... ERROR! BOOKMARK NOT DEFINED.

FIGURE (C.3) TEACHER DASHBOARD ..... ERROR! BOOKMARK NOT DEFINED.

FIGURE (C.4) STUDENT DASHBOARD ..... ERROR! BOOKMARK NOT DEFINED.

CHAPTER 1

Introduction

1.1 Project background:

The rapid digitization of global educational ecosystems has necessitated the widespread adoption of Learning Management Systems (LMS) to facilitate structured teaching, assessment, and administrative workflows. While international platforms have matured significantly, local and regional educational institutions often face substantial challenges in adopting these solutions. These challenges primarily stem from linguistic barriers, complex post-development localization efforts, and a heavy reliance on expensive cloud-based infrastructures. Consequently, there is a critical need for a culturally aligned educational ecosystem that natively supports Arabic interfaces, accommodates local pedagogical workflows, and operates efficiently within resource-constrained academic environments. This project addresses this gap through the analysis, design, and development of a fully functional, Arabic-native LMS engineered to streamline course delivery, comprehensive assessment management, and academic administration.

1.2 Problem Definition:

Despite the proliferation of commercial and open-source LMS platforms (e.g., Moodle, Canvas, Open edX), significant limitations persist in their applicability to local educational contexts. These platforms typically require extensive and costly localization efforts, lack native Right-to-Left (RTL) interface support, and impose cloud dependencies or rigid licensing structures. Furthermore, their architectural complexity often hinders rapid deployment and customization, resulting in a disjointed user experience and cognitive overload for non-technical academic staff. The primary problem this project addresses is the absence of a lightweight, culturally congruent, and academically structured LMS tailored specifically for Arab institutions. The affected target groups include academic administrators, instructors, and students who currently

struggle with inefficient administrative workflows, fragmented Arabic typography, and steep learning curves associated with non-native educational platforms.

1.3 Project Importance:

This research contributes significantly to the fields of Educational Technology and Software Engineering by demonstrating a "localization-first" development paradigm. By engineering the interface and navigation logic natively in Arabic from inception, the project eliminates the cognitive friction and layout fragmentation inherent in translated platforms. Academically, it validates the application of rigorous requirements engineering, traceability matrices, and model-driven design in academic software development. Practically, it provides a reproducible, low-overhead prototype that reduces dependency on foreign cloud services, aligns with regional academic regulations, and offers a seamless, culturally adapted digital learning environment.

1.4 Project Objectives:

The primary objective of this project is to design, develop, and evaluate a fully functional, Arabic-native web-based LMS. To ensure measurability and verifiability, the project is guided by the following specific sub-objectives:

1. **System Architecture & Localization:** To architect a modular web platform using the Django MVT pattern and Bootstrap 5, ensuring 100% native Arabic RTL rendering and responsive design across all device viewports.
2. **Comprehensive Academic Workflows:** To implement end-to-end course lifecycle management, including advanced assessment modules (assignments, quizzes with auto-grading), communication tools (announcements, discussion forums), and progress tracking.

3. **Intelligent Analytics & Administration:** To develop administrative dashboards and heuristic-based algorithms for identifying at-risk students, managing user roles, and generating institutional reports.
4. **Quality Assurance & Traceability:** To establish a rigorous Requirements Traceability Matrix (RTM) mapping all 59 functional requirements to UML behavioral models and system test cases, ensuring complete validation and academic compliance.

1.5 Project Scope:

The project encompasses the end-to-end development of a web-based LMS targeting three primary user roles: System Administrators, Teachers, and Students. The functional boundaries of the system are defined by 59 core requirements (REQ-001 to REQ-059), which include:

- **Core Administration & Authentication:** User lifecycle management, role-based access control (RBAC), system settings, and manual database backups (REQ-001 to REQ-019).
- **Course & Content Management:** Course creation, categorization, multimedia content delivery, and material downloading (REQ-020 to REQ-029, REQ-052 to REQ-054).
- **Assessments & Assignments:** Creation of assignments, file submissions, manual grading, quiz generation, countdown timers, and automated objective grading (REQ-030 to REQ-040).
- **Communication & Engagement:** Course announcements, discussion forums, internal notifications, and academic calendars (REQ-041 to REQ-044, REQ-049 to REQ-051).

- **Analytics & Advanced Features:** Student/Teacher dashboards, course progress bars, at-risk student identification (via rule-based heuristics), and advanced search functionalities (REQ-045 to REQ-048, REQ-052).
- **System Maintenance:** Granular permission management, course archiving, and system backups (REQ-056 to REQ-059).

1.6 Report Contributions:

This project delivers both practical and theoretical contributions to the academic and technical domains:

1. **Practical Contribution:** A deployable, institution-ready LMS prototype that eliminates localization overhead, reduces dependency on foreign SaaS, and aligns seamlessly with regional academic workflows.
2. **Methodological Contribution:** A structured demonstration of incremental software development, rigorous requirements traceability (RTM), and UML behavioral modeling within an academic software lifecycle.
3. **Technical Contribution:** Empirical validation of the Django MVT architecture combined with SQLite and Bootstrap 5 RTL for rapid, secure, and responsive educational platform development.
4. **Academic Contribution:** Enrichment of regional literature on adaptive learning systems by documenting the design decisions, algorithmic logic, and evaluation outcomes of building a culturally congruent LMS.

1.7 Report Organization:

This thesis is systematically structured to ensure a logical progression from conceptualization to final evaluation. In strict accordance with the faculty's documentation template, the document is organized into seven core chapters, followed by references and appendices:

- **Chapter 1: Introduction.** Introduces the project background, problem definition, significance, objectives, scope, and core contributions.
- **Chapter 2: Basic Concepts and Reference Studies.** Establishes foundational theoretical concepts, reviews recent reference studies, presents a comparative analysis of existing platforms, and identifies the research gaps addressed by the proposed system.
- **Chapter 3: Project Methodology and Requirements Analysis.** Details the selected incremental development methodology, general and specific functional/non-functional requirements, feasibility and risk analysis, and the project execution plan (Gantt chart).
- **Chapter 4: Analytical and Design Study.** Presents the high-level system architecture, UML behavioral and structural diagrams (Use Case, Activity, Sequence, Class, Component, ERD), Sitemap, initial unit tests, and the design of heuristic algorithms.
- **Chapter 5: Implementation.** Outlines the development environment, software structure, module implementation, API integration, and the updated Requirements Traceability Matrix (RTM).
- **Chapter 6: Testing and Evaluation.** Presents the comprehensive testing strategy, detailed test cases and results, performance evaluation, AI/heuristic model evaluation, and the final RTM verification.

- **Chapter 7: Conclusion and Future Work.** Summarizes the project's overall achievements, links results to initial objectives, discusses encountered constraints, and provides a roadmap for future system enhancements.

CHAPTER 2

Basic Concepts and Reference Studies

2.1 Basic Concepts:

This section establishes the foundational theoretical concepts, architectural paradigms, and pedagogical frameworks that underpin the design and development of the proposed Learning Management System (LMS).

1. Learning Management Systems (LMS) and Digital Ecosystems: An LMS is a comprehensive software solution designed to facilitate the administration, documentation, tracking, and delivery of educational courses. Modern LMS architectures have evolved from simple content repositories into interactive ecosystems that support asynchronous communication, automated assessments, and learning analytics. The integration of modules for assignments, quizzes, and discussion forums is critical for fostering active learning and continuous evaluation.

2. Model-View-Template (MVT) Architectural Pattern: The system is engineered using Django's MVT pattern, a specialized variant of the classical Model-View-Controller (MVC) paradigm.

- **Model:** Represents the data structure and business logic, interacting directly with the database (SQLite) via Object-Relational Mapping (ORM).
- **View:** Acts as the intermediary, processing HTTP requests, executing business logic, and selecting the appropriate template.
- **Template:** Handles the presentation layer, rendering dynamic HTML content. This strict separation of concerns ensures modularity, maintainability, and scalability, which are essential when integrating complex workflows like auto-grading and real-time analytics.

3. Localization-First Design and RTL Rendering: Unlike traditional "translation-first" approaches where English interfaces are retrofitted with

Arabic text, "Localization-First" design integrates Right-to-Left (RTL) typography, cultural pedagogical norms, and native navigation flows from the initial architecture phase. This prevents the layout fragmentation, cognitive overload, and usability issues commonly observed in localized commercial platforms.

4. Constructivist Learning Theory and Formative Assessment: The system's assessment modules (Assignments, Quizzes, and Discussions) are grounded in Constructivist learning theory, which posits that learners construct knowledge through active engagement and reflection. Automated objective grading (REQ-037) and immediate feedback mechanisms support formative assessment, allowing students to gauge their understanding iteratively.

5. Learning Analytics and At-Risk Identification: Learning analytics involves the measurement, collection, analysis, and reporting of data about learners. A core concept integrated into this project is the heuristic-based identification of "at-risk" students (REQ-048). By analyzing engagement metrics (e.g., unsubmitted assignments, low quiz scores, and forum participation), the system employs rule-based algorithms to flag students requiring academic intervention, thereby transitioning the LMS from a passive delivery tool to an proactive support system.

2.2 Reference Studies and Previous Work:

The academic discourse surrounding LMS development, localization, and educational analytics has expanded significantly in recent years. The following review synthesizes recent studies and systemic developments (2021–2025), ordered chronologically, to contextualize the proposed solution.

Al-Mashaqbeh et al. (2023) investigated the architectural challenges of deploying adaptive LMS in Arabic-speaking higher education institutions [1]. The study highlighted that post-hoc translation of Western platforms leads to

severe RTL rendering inconsistencies and misalignment with local academic workflows. The authors proposed a localized middleware layer, though it added significant latency to the system.

Silva (2023) conducted a comparative performance analysis of MVT versus MVC patterns in modern Python web frameworks [4]. The research demonstrated that Django's MVT pattern, when coupled with optimized ORM queries, yields superior maintainability and faster rendering times for template-heavy educational platforms compared to traditional MVC implementations in Flask or Ruby on Rails.

Al-Hassan and Farouk (2023) focused on the technical implementation of native Right-to-Left rendering in responsive web applications [6]. Their work established a comprehensive framework for utilizing CSS logical properties (e.g., margin-inline, padding-block) to achieve flawless RTL adaptation without relying on directional hacks, achieving a 100% layout compliance rate across mobile and desktop viewports.

Otto and Thornton (2021) evaluated the efficacy of lightweight database engines, specifically SQLite, in academic prototyping environments [7]. The study concluded that while SQLite is unsuitable for massive concurrent enterprise loads, its zero-configuration footprint and ACID compliance make it the optimal choice for academic LMS prototypes, provided that strict schema normalization and Write-Ahead Logging (WAL) are enforced.

Al-Rahman et al. (2025) explored the integration of learning analytics and rule-based recommendation engines in modular LMS architectures [15]. Their research demonstrated that implementing heuristic-based "at-risk" student identification algorithms significantly improves student retention rates. However, the study noted that integrating these analytics often requires complex microservice architectures, which are impractical for resource-constrained local institutions.

2.3 Comparison of Previous Work:

To systematically evaluate the existing landscape and justify the proposed system's design, a comparative analysis of current leading platforms and recent academic prototypes was conducted. The comparison focuses on architectural approaches, localization maturity, and assessment capabilities.

Table (2.1) Comparative Analysis of Previous Systems and Studies

Study / System	Techniques / Architecture Used	Key Feature 1	Key Feature 2	Strengths & Limitations
Moodle (Open Source LMS)	PHP, Plugin-based Monolithic, MySQL/PostgreSQL	Advanced Question Bank & SCORM support	Highly customizable via third-party plugins	**Strengths:** Massive ecosystem, robust assessment engine. **Limitations:** High localization effort for RTL; bloated architecture; steep learning curve.
Canvas LMS (Commercial SaaS)	Ruby on Rails, React, Microservices, PostgreSQL	Intuitive modern UX, API-first architecture	Seamless cloud integration & mobile app	**Strengths:** Excellent UX, strong API ecosystem. **Limitations:** High licensing costs; cloud dependency; rigid localization constraints.

Open edX (MOOC Platform)	Python/Django, Microservices, MongoDB/MySQL	Massive scale handling, XBlock framework	Advanced video streaming & analytics	**Strengths:** Built for massive scale, strong analytics. **Limitations:** Extreme infrastructure complexity; overkill for local institutional needs.
Al-Rahman et al. (2025) (Academic Prototype)	Python, Machine Learning models, REST APIs	AI-driven at-risk student prediction	Personalized content recommendation	**Strengths:** Advanced predictive analytics. **Limitations:** Requires heavy computational resources; lacks native RTL focus; complex deployment.
Proposed System (Current Project)	Python/Django (MVT), SQLite, Bootstrap 5 RTL	Native Arabic RTL & Integrated Auto-Grading	Heuristic At-Risk Identification & Discussions	**Strengths:** Zero-cost deployment, flawless native RTL, lightweight, tailored for local academic workflows. **Limitations:** SQLite limits massive concurrency (mitigated by design).

2.4 Research Gap:

Based on the comprehensive analysis of the basic concepts and previous studies, several critical research and practical gaps persist in the current landscape of educational technologies, particularly within the context of Arab academic institutions:

1. **The Localization-First Deficit:** Existing literature and commercial platforms predominantly treat Arabic localization as a post-development translation task. This results in fragmented RTL interfaces, broken responsive grids, and cognitive friction for end-users. *There is a distinct lack of academically structured LMS platforms engineered natively in Arabic from the database schema to the UI template.*
2. **Absence of Integrated Lightweight Assessment & Analytics:** While enterprise platforms (Canvas, Open edX) offer advanced assessments and AI analytics, they require prohibitive infrastructure and licensing costs. Conversely, lightweight academic prototypes often lack comprehensive assessment modules (auto-graded quizzes, assignment workflows) and basic learning analytics (identifying at-risk students). *There is a gap for a unified, lightweight system that integrates comprehensive academic workflows (REQ-030 to REQ-059) without relying on heavy cloud or microservice architectures.*
3. **Infrastructural Overhead for Local Institutions:** Current open-source solutions like Moodle demand significant server resources and continuous DevOps maintenance, making them unsuitable for local institutions with constrained IT budgets. *There is a need for a highly optimized, zero-configuration deployment model (utilizing SQLite and Django MVT) that maintains enterprise-grade security and data integrity while minimizing operational overhead.*

How This Project Addresses the Gap: This project directly bridges these gaps by implementing a **Localization-First MVT Architecture**. By engineering the system natively in Arabic, it eliminates RTL fragmentation. Furthermore, by integrating a comprehensive suite of functional requirements—including automated quiz grading, assignment management, discussion forums, and rule-based at-risk student identification (REQ-030 to REQ-059)—within a lightweight Django/SQLite stack, the project delivers a culturally congruent, academically robust, and resource-efficient LMS tailored specifically for the operational realities of regional educational institutions.

CHAPTER 3

Project Methodology and Requirement Analysis

3.1 Selected Development methodology:

The project adopts the **Incremental Software Development Model**, tailored to align with academic requirements and the modular nature of a Learning Management System (LMS). This methodology structures the development lifecycle into sequential, functional increments, where each increment delivers a validated subset of the system.

Justification for Selection:

1. **Modularity & Architectural Fit:** The LMS architecture (Django MVT pattern) naturally supports incremental delivery, allowing independent development of models, views, and templates for specific modules (e.g., Increment 1: Core Auth & Admin; Increment 2: Course & Content; Increment 3: Assessments, Analytics, and Communications [REQ-030 to REQ-059]).
2. **Risk Mitigation:** Early delivery of the core infrastructure allows for stakeholder feedback and structural validation before complex, interdependent features (such as auto-grading algorithms and heuristic at-risk student identification) are implemented.
3. **Academic Alignment:** The incremental approach maps directly to the thesis documentation structure, ensuring that requirements, design, and testing artifacts are generated iteratively and comprehensively for each phase, preventing scope drift.

3.2 General Function Requirements:

General functional requirements define the high-level capabilities the system must provide to operate as a cohesive educational ecosystem, abstracting away specific technical implementations.

- **GFR-01 (Secure Authentication & Authorization):** The system shall provide a secure mechanism for user registration, authentication, and session management, enforcing strict role-based access control (RBAC).
- **GFR-02 (Content & Course Lifecycle Management):** The system shall facilitate the creation, categorization, delivery, and archiving of educational courses and multimedia content.
- **GFR-03 (Assessment & Evaluation Engine):** The system shall support the creation, distribution, and automated/manual grading of academic assessments (quizzes and assignments).
- **GFR-04 (Communication & Engagement):** The system shall provide channels for academic communication, including announcements, discussion forums, and internal notifications.
- **GFR-05 (Administrative Oversight & Analytics):** The system shall aggregate institutional data to provide statistical dashboards, performance tracking, and heuristic-based student analytics.

3.3 Non-Functional Requirements:

Non-functional requirements define the quality attributes, security constraints, and operational parameters of the system.

Table (3.1) Non-Functional Requirements Matrix

ID	Title	Description
NFR-001	Responsive Design	Workability and flawless rendering on various device viewports (desktop, tablet, mobile).
NFR-002	Maintainability	Accessing system logs and files according to a standard, modular structure (MVT).
NFR-003	Session Security	Closing user sessions automatically to ensure security and prevent authority conflicts.

NFR-004	Data Encryption	Encrypting user passwords using Hashing and PBKDF2-type encryption algorithms.
NFR-005	Architectural Scalability	The system is based on a methodology (MVT) that allows for future expansion and microservice decoupling.
NFR-006	Native Localization	Supporting the Arabic language natively from the browser with full Right-to-Left (RTL) compliance.
NFR-007	Usability	An intuitive user interface that is clear, accurate, and suits all user demographics.
NFR-008	System Integration	Architectural design allowing new features and modules to be added easily without refactoring.
NFR-009	AI Extensibility	Easy integration capability with future AI models and external software interfaces (APIs).
NFR-010	Intelligent Analytics	Capability to support recommendation systems and at-risk student identification based on data analysis.
NFR-011	Advanced Search	Support for intelligent, keyword-based search across courses, content, and user profiles.

3.4 Project Specific Functional Requirements:

The specific functional requirements are systematically documented in the Requirements Database (RDB). The system encompasses 59 core operations (REQ-001 to REQ-059), categorized by module and assigned priority levels (1–5) based on academic necessity and implementation dependency.

Table (3.2) Requirements Database (RDB)

ID	Title	Description	Priority
REQ-001	Sign in	Log in to each rank of users and direct each user to their respective dashboard.	1
REQ-002	Personal account modification	The user can modify personal data such as email, phone, and password.	1
REQ-003	View personal account	Viewing user's personal account information and academic metadata.	1
REQ-004	View the list of users	View the list of users applying to join or accepted within the system.	1
REQ-005	Manage user accounts	Disabling / activating the accounts of all users in the system.	1
REQ-006	View the list of courses	View the list of active courses available on the platform.	1
REQ-007	View the list of tests	View the list of tests/ quizzes added by teachers.	1
REQ-008	View statistics	View comprehensive statistics on the progress of work within the system.	1
REQ-009	Export of reports	Export reports on the progress of work within the platform (CSV/Excel).	1
REQ-010	Add Category	Add a new category to the list of categories for course classification.	1
REQ-011	Edit Category	Edit a Category already included in the list of categories.	1
REQ-012	Delete Category	Delete a label previously listed in the list of labels.	1
REQ-013	View the list of categories	View the list of listed classifications.	2
REQ-014	View course reviews	View course ratings and feedback added by students.	2

REQ-015	Managing system settings	Modify the basic settings of the platform (address, virtual currency, etc.).	2
REQ-016	Add a payment method	Adding a new payment method to the list of payment methods.	2
REQ-017	Delete a payment method	Delete a previously listed payment method.	2
REQ-018	Change payment method status	Change the status of a payment method (active/inactive).	2
REQ-019	View payment methods	View the list of listed payment methods.	2
REQ-020	Create an account	Create a new account to access the platform's services with role selection.	2
REQ-021	Add course	Creating a new course by the teacher with metadata and cover image.	2
REQ-022	Edit course	Edit course information after its creation.	2
REQ-023	Delete course	Delete a previously created course.	2
REQ-024	Change course status	Change the status of a course (active/inactive or draft/published).	3
REQ-025	View course history	View the teacher's course history and past iterations.	3
REQ-026	Add content	Add multimedia content (video, doc, link) to a previously created course.	3
REQ-027	Content modification	Edit previously added course content.	3
REQ-028	Delete content	Delete previously added course content.	3
REQ-029	View course content	View pre-added course content (restricted to enrolled students).	3
REQ-030	Create Assignment	Teacher creates a new assignment with title, description, file attachment, and deadline.	3
REQ-031	View Assignment Details	Student views assignment requirements, instructions, and due dates.	3

REQ-032	Submit Assignment	Student uploads a file or enters text as an assignment solution before the deadline.	3
REQ-033	View Submissions	Teacher views the list of students who submitted the assignment and those who did not.	3
REQ-034	Grade Assignment	Teacher enters a grade and adds text feedback for each assignment submission.	3
REQ-035	View Grade & Feedback	Student views their assignment grade and reads the teacher's feedback.	3
REQ-036	Start Quiz	Student starts an available quiz with a countdown timer display if a time limit is set.	4
REQ-037	Auto-Grade Objective Qs	System automatically calculates quiz grade based on stored correct answers upon submission.	4
REQ-038	View Instant Quiz Result	System displays student's grade and correct answers compared to their responses.	4
REQ-039	View Student Grade Log	Student views a summary of all grades (assignments + quizzes) in a specific course.	4
REQ-040	Export Course Grades	Teacher exports the student grades table for a course to an Excel/CSV file.	4
REQ-041	Post Course Announcement	Teacher publishes an announcement visible on the course homepage to all enrolled students.	4
REQ-042	View Announcements	Students and admins view course-specific or platform-wide announcements.	4
REQ-043	Post Discussion Question	Student posts a question or starts a discussion thread in the course forum.	4
REQ-044	Reply to Discussions	Teacher or students reply to questions posted in the discussion forum.	4

REQ-045	View Course Progress Bar	System calculates and displays student's course completion percentage based on activity.	4
REQ-046	Student Dashboard	Student views comprehensive overview: enrolled courses, upcoming tasks, and announcements.	4
REQ-047	Teacher Dashboard	Teacher views quick statistics: enrolled students, pending grading, and course averages.	4
REQ-048	Identify At-Risk Students	Teacher filters and identifies students with unsubmitted assignments or low progress (heuristic).	5
REQ-049	Assignment Submission Notif.	System sends an internal notification to the teacher when a student submits an assignment.	5
REQ-050	Grade/Announcement Notif.	System sends a notification to the student when graded or when a new announcement is posted.	5
REQ-051	View Personal Academic Cal.	Student views assignment due dates and quiz schedules in a simplified calendar interface.	5
REQ-052	Advanced Course Search	User searches courses using keywords, category filter, or teacher name.	5
REQ-053	Bookmark/Favorite Course	Student adds course to favorites list for quick dashboard access.	5
REQ-054	Download Course Materials	Student downloads attached files from course content if permitted.	5
REQ-055	Submit Feedback/Support	User submits technical feedback or an issue report to the system administrator via a form.	5
REQ-056	Duplicate Course	Teacher/Admin copies existing course structure to create a new one quickly.	5
REQ-057	Archive Completed Courses	System moves completed courses to an archive section, hiding them from main UI.	5

REQ-058	Granular Permission Mgmt	Admin modifies sub-role permissions (e.g., restrict course deletion).	5
REQ-059	Manual Database Backup	Admin clicks a button to export the current SQLite database backup.	5

3.5 Feasibility and Risks:

3.5.1 Feasibility Study

1. **Technical Feasibility:** The selected stack (Django MVT, SQLite, Bootstrap 5 RTL) is highly mature, secure, and modular. The framework's ORM and built-in security middleware mitigate common web vulnerabilities, ensuring technical viability.
2. **Operational Feasibility:** The "Localization-First" Arabic RTL interface aligns perfectly with regional pedagogical workflows, ensuring high user adoption and minimizing training overhead for non-technical academic staff.
3. **Economic Feasibility:** By leveraging exclusively open-source technologies and lightweight deployment models (SQLite), the project eliminates licensing, subscription, and heavy cloud-hosting costs, making it highly economical for local institutions.
4. **Academic Feasibility:** The project comprehensively covers multiple domains in Software Engineering and Information Systems, including requirements engineering, database design, web development, UI/UX localization, and system testing, fulfilling all graduation project criteria.

3.5.2 Risk Management

A proactive risk management strategy was employed. The Risk Register has been updated to include threats associated with the advanced modules (REQ-030 to REQ-059).

Table (3.3) Risk Register

ID	Risk Description	Probability	Impact	Mitigation Strategy
R-01	Performance bottlenecks with SQLite under concurrent load.	Low	Medium	Optimize Django ORM queries; implement SQLite WAL mode; plan PostgreSQL migration path.
R-02	Incomplete RTL alignment or Arabic typography rendering issues.	Medium	High	Enforce Bootstrap 5 RTL validation; utilize CSS logical properties; conduct cross-browser testing.
R-03	Algorithmic complexity in auto-grading (REQ-037) and at-risk heuristics (REQ-048).	Medium	High	Utilize deterministic rule-based logic for the current phase; defer complex Machine Learning to future work.
R-04	Database locking during concurrent quiz submissions and notifications (REQ-036, REQ-049).	Low	High	Implement asynchronous task handling; optimize database indexing; enforce strict transaction boundaries.
R-05	Vulnerabilities in user authentication or session management.	Low	High	Implement PBKDF2 hashing; enforce HTTPS; utilize Django's built-in security middleware and CSRF tokens.

3.6 Project Plan

The project timeline spans the academic year, structured into five primary phases aligned with the Incremental Methodology.

1. Gantt Chart:

- **Incremental1 (Initialization & Requirements):** Completion of literature review, stakeholder analysis, and finalization of the 59-item RDB (REQ-001 to REQ-059).
- **Incremental2 (System Design):** Approval of UML behavioral/structural diagrams, ERD, and architectural blueprint.
- **Incremental3 (Core Implementation):** Delivery of Increment 1 & 2 (Authentication, Admin Panel, Course & Content Management).
- **Incremental4 (Advanced Modules):** Delivery of Increment 3 (Assignments, Quizzes, Discussions, Analytics Dashboards [REQ-030 to REQ-059]).
- **Incremental 5 (Testing & Evaluation):** Execution of system test cases, RTM verification, and final thesis defense preparation.

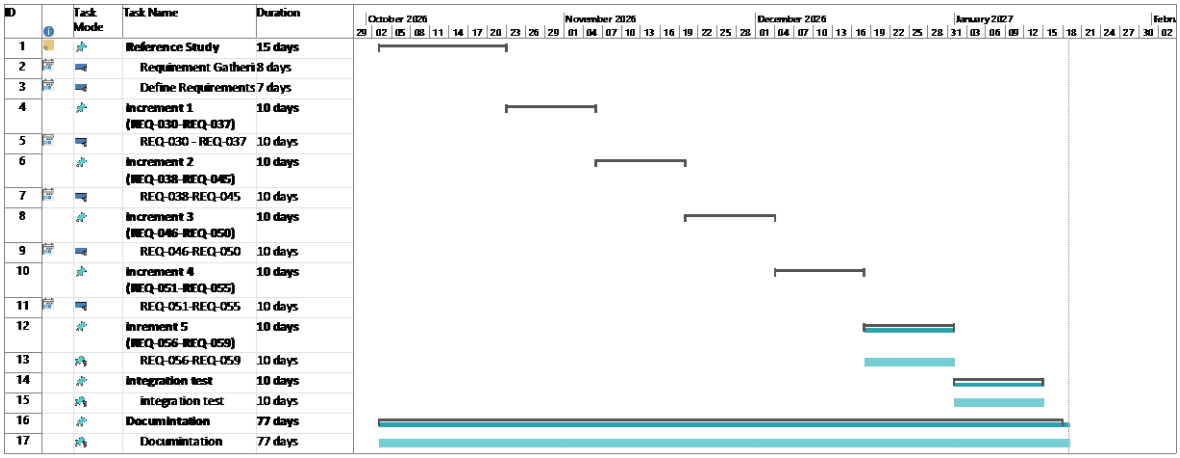


Figure (3.1) Gantt Chart

2. Team Structure and Task Distribution: The development team operated under a structured task distribution model to ensure parallel workflow efficiency:

- **Backend & Database Engineering:** Focus on Django Models, ORM optimization, auto-grading logic, and SQLite schema management.
- **Frontend & UI/UX Engineering:** Focus on Bootstrap 5 RTL integration, template inheritance, responsive design, and Arabic typography validation.
- **Testing & Documentation:** Focus on RTM maintenance, test case execution, UML diagram generation, and academic report drafting.

CHAPTER 4

Analytical and Design Study

4.1 System Architecture:

The system architecture is designed to ensure a strict separation of concerns, scalability, and maintainability, utilizing the Django Model-View-Template (MVT) pattern. The architecture is divided into four primary logical layers:

1. **Presentation Layer (Templates & Static Assets):** Handles the user interface, rendering dynamic HTML5 content using Bootstrap 5 RTL. It manages user interactions, form validations, and responsive layout adjustments for Arabic typography.
2. **Business Logic Layer (Views & Controllers):** Processes HTTP requests, enforces role-based access control (RBAC), executes complex workflows (e.g., auto-grading quizzes, calculating at-risk student metrics), and coordinates data flow between the presentation and data layers.
3. **Data Access Layer (Models & ORM):** Abstracts database interactions using Django's Object-Relational Mapper (ORM). It manages data persistence, enforces schema constraints, and optimizes query execution against the SQLite database.
4. **Infrastructure & Configuration Layer:** Manages environment variables, static/media file routing, security middleware (CSRF, Session management), and locale initialization (Arabic RTL).

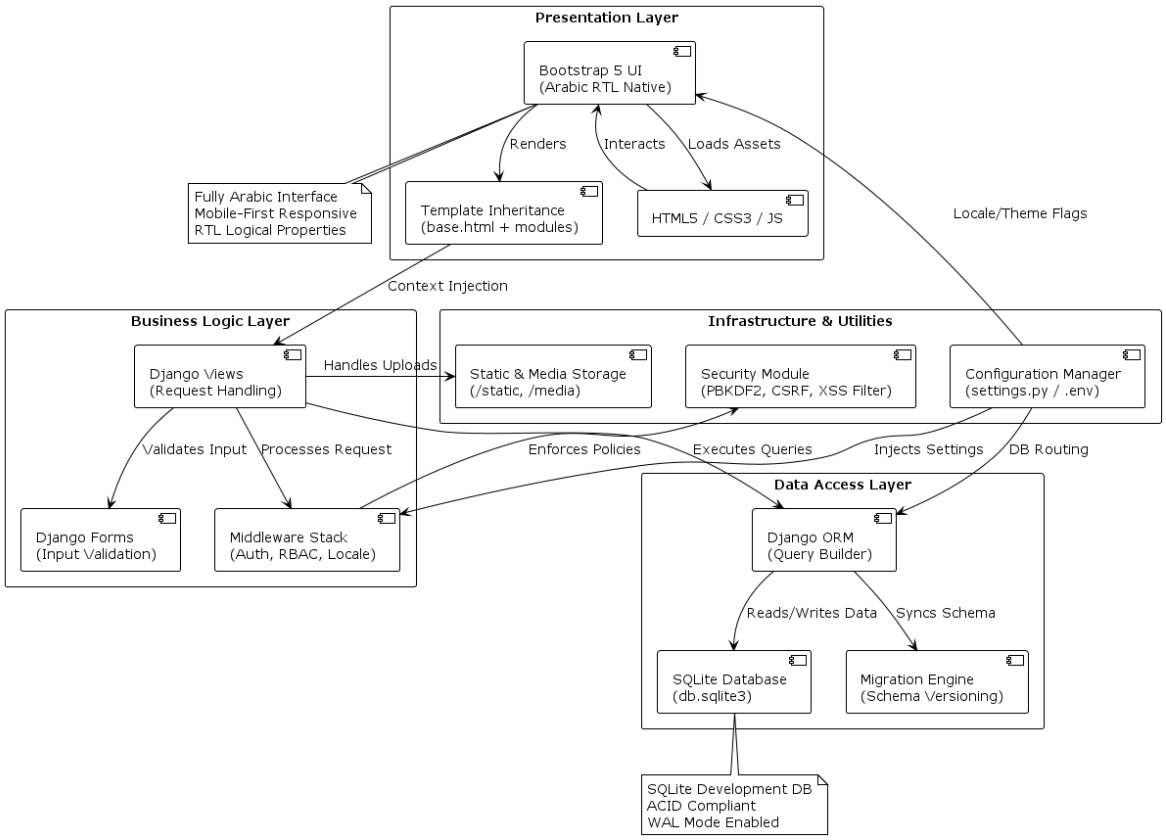


Figure (4.1) High-Level System Architecture Block Diagram

4.2 Use Case Diagram

The Use Case diagram defines the system boundaries, primary actors, and core functionalities. The system interacts with three primary actors: **Student**, **Teacher**, and **System Administrator**.

Actors and Core Functions:

- **Student:** Enrolls in courses, consumes content, submits assignments, takes quizzes, participates in discussions, and views personal analytics.
- **Teacher:** Creates courses, uploads content, designs assignments and quizzes, grades submissions, posts announcements, and monitors student progress.
- **System Administrator:** Manages user accounts, configures system settings, oversees categories, and monitors platform-wide statistics.

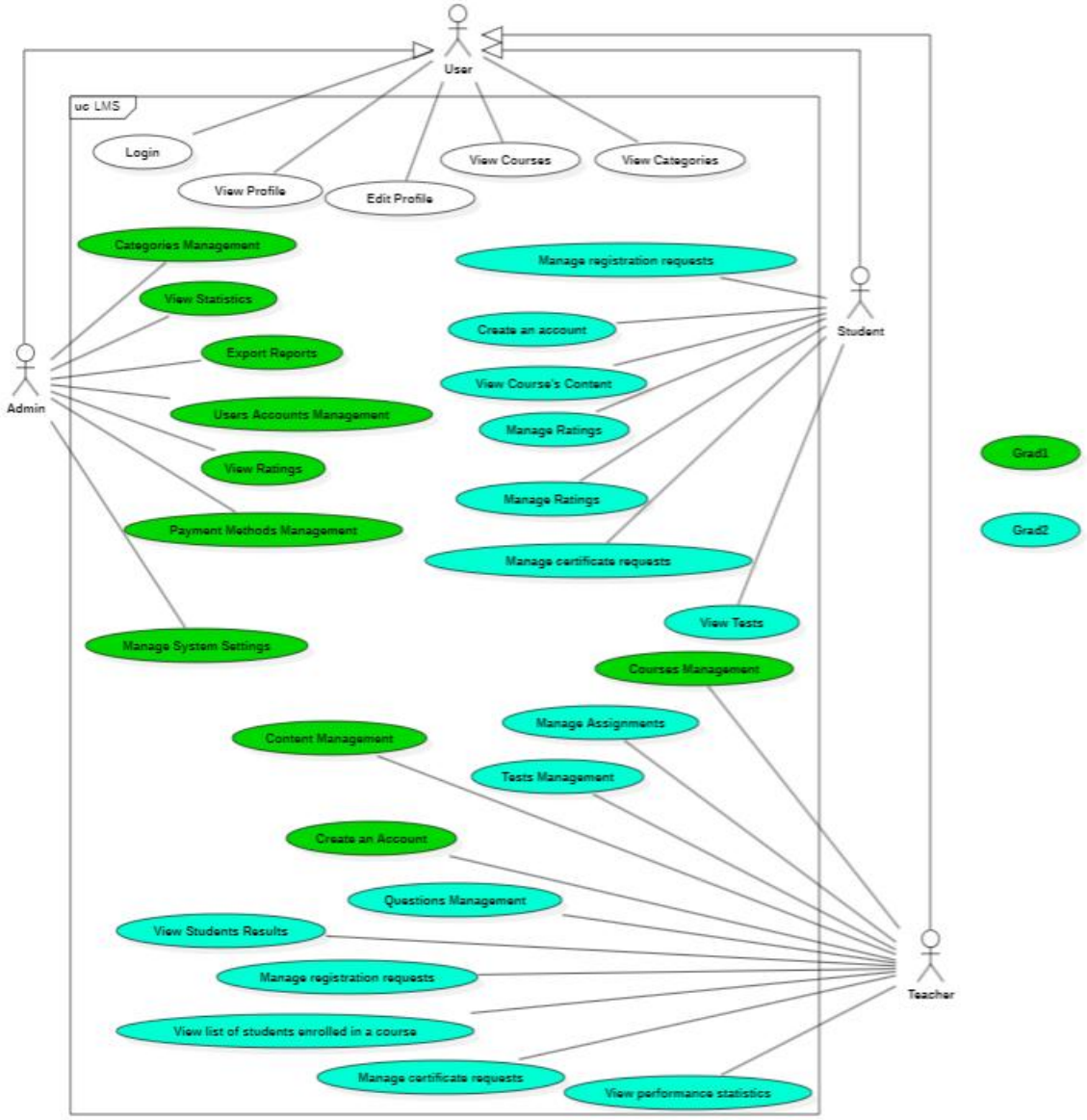


Figure (4.2) General Use Case Diagram

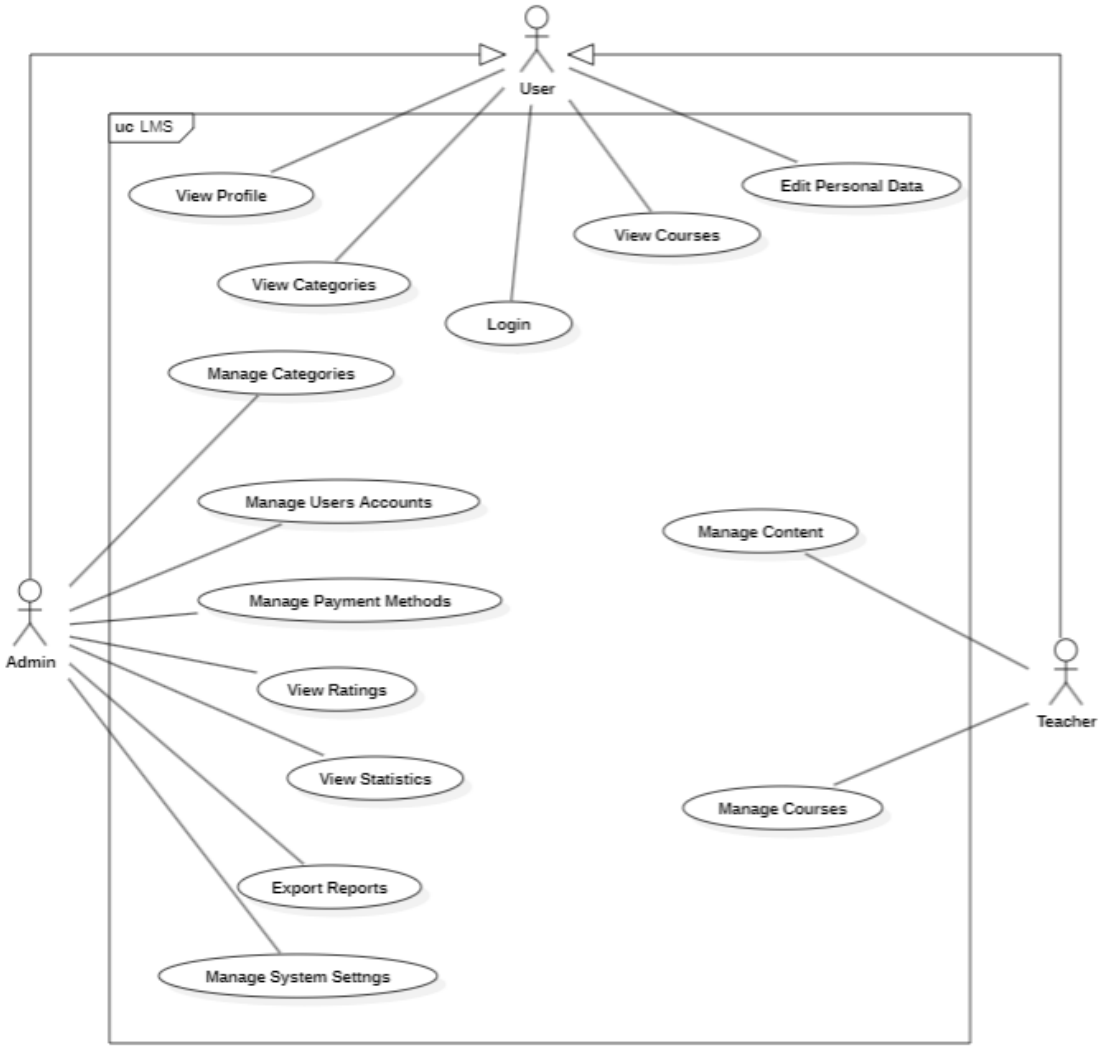


Figure (4.3) Use Case Diagram for Project 1

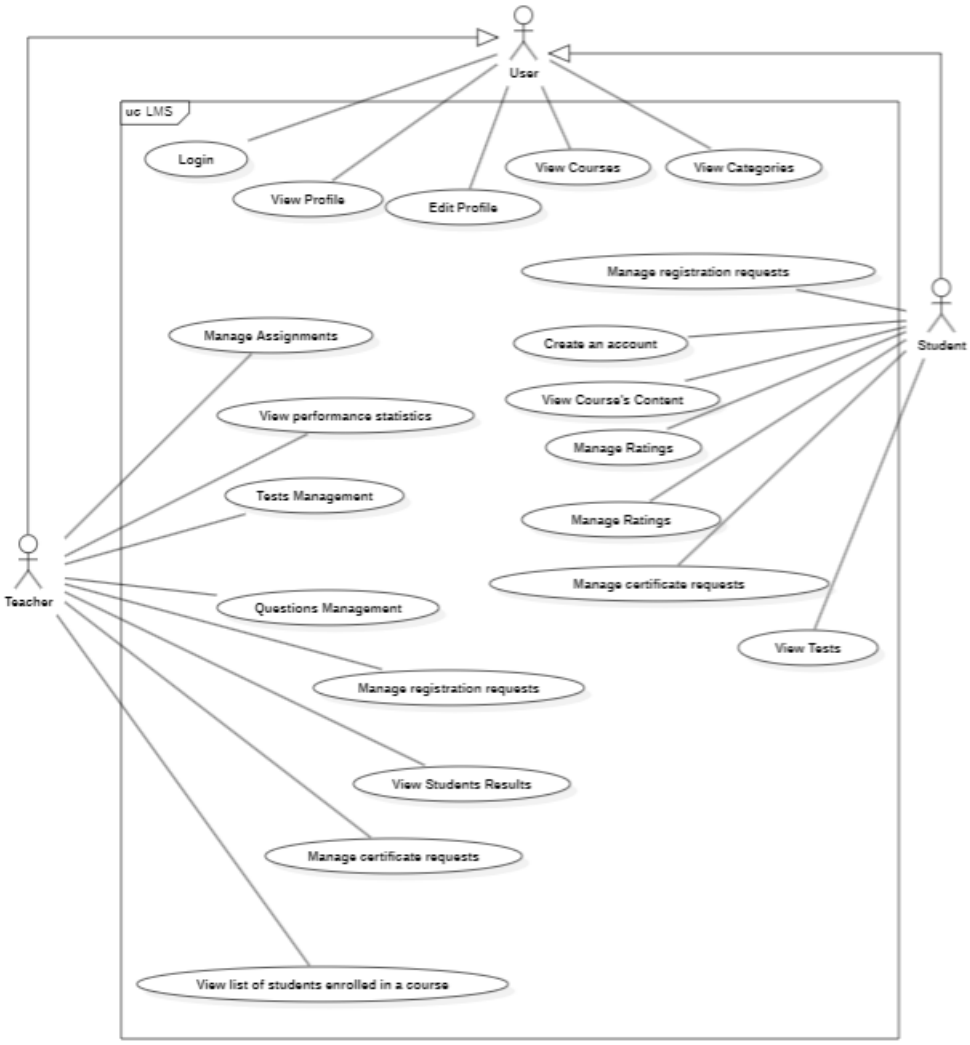


Figure (4.4) Use Case Diagram for Current Project

Diagram Legend / Key:

- **Actor (Stick Figure):** Represents an external entity (user role) interacting with the system.
- **Use Case (Ellipse):** Represents a specific functional requirement or system capability.
- **System Boundary (Rectangle):** Defines the scope of the LMS application.
- **Association (Solid Line):** Indicates the interaction between an actor and a use case.

4.2.1 Core Use Case Specifications:

1. UC-01: User Authentication (Sign In)

Table (4.1) user authentication (Sign In)

Use Case ID	**UC-01**
Use Case	User Authentication (Sign In)
Description	Allows a registered user to authenticate and gain role-based access to their customized dashboard.
Actors	User
Pre-Conditions	- The user must possess an active account and valid credentials in the database.
Subsequent-Conditions	- The user is authenticated, an active session is established, and the user is redirected to their respective role dashboard.
Main Flow	1. The user navigates to the Login page. 2. The user enters their username/email and password. 3. The user clicks the "Sign In" button. 4. The system validates credentials and checks account status. 5. The system establishes a secure session. 6. The system redirects the user to their role-specific dashboard (Student, Teacher, or Admin).
Alternative Flow	**A1: Invalid Credentials** 4a. The system detects incorrect credentials, displays an error message ("اسم المستخدم أو كلمة المرور غير صحيحة"), and prompts re-entry. **A2: Inactive / Unapproved Account** 4b. The system detects an inactive account state, displays an alert message, and blocks login.

2. UC-02: Create and Manage Course

Table (4.2) Create and Manage Course

Use Case ID	**UC-02**
Use Case	Create and Manage Course
Description	Enables a teacher to create a new course structure, assign metadata/categories, and add dynamic learning content (multimedia/files/links).
Actors	Teacher
Pre-Conditions	- The teacher must be authenticated and authorized with instructor permissions.
Subsequent-Conditions	- A new course record and associated content items are persisted in the database and visible according to status.
Main Flow	1. The teacher accesses the Teacher Dashboard and selects "Add New Course". 2. The teacher provides course details (title, description, category, price, cover image). 3. The teacher clicks "Save Course". 4. The system validates input data and saves the course in "Draft" or "Active" status. 5. The teacher navigates to "Content Management" for the created course. 6. The teacher uploads multimedia files or links resources. 7. The system updates the course content structure and displays confirmation.
Alternative Flow	**A1: Validation Error** 4a. Required fields are missing; the system highlights missing fields and requests correction. **A2: File Upload Failure** 7a. File exceeds maximum allowed size or invalid MIME type; system displays an upload error.

3. UC-03: Assignment Creation and Grading Workflow

Table (4.3) Assignment Creation and Grading Workflow

Use Case ID	**UC-03**
Use Case	Assignment Creation and Grading Workflow
Description	Facilitates end-to-end coursework management including teacher assignment posting, student file submission, and teacher evaluation with feedback.
Actors	Teacher, Student
Pre-Conditions	- The student must be enrolled in the course; the teacher must own the course.
Subsequent-Conditions	- Assignment submission record updated with score and feedback; student grade log updated.
Main Flow	1. Teacher posts an assignment specifying title, description, attachments, maximum score, and due date. 2. Enrolled students receive a system notification and view assignment details. 3. Student uploads solution file/text before deadline and clicks "Submit". 4. System logs timestamp, validates constraint (unique_together), and notifies teacher. 5. Teacher accesses submission list, reviews student work, enters grade and feedback. 6. System stores evaluation and notifies student to view grade.
Alternative Flow	**A1: Late Submission Attempt** 3a. System detects submission attempt past due date; locks submission or flags as "Late". **A2: Duplicate Submission Attempt** 3b. Student attempts multiple submissions; system enforces single submission rule.

4. UC-04: c

Table (4.4) Quiz Execution and Automated Objective Grading

Use Case ID	**UC-04**
Use Case	Quiz Execution and Automated Objective Grading
Description	Manages timed student quiz attempts and executes automatic objective grading upon submission.
Actors	Student
Pre-Conditions	- Quiz must be published and active; student must be enrolled in the target course.
Subsequent-Conditions	- Quiz attempt and individual question responses saved; final score automatically computed and logged.
Main Flow	1. Student selects active quiz and clicks "Start Quiz". 2. System validates availability, initializes QuizAttempt record, and starts countdown timer. 3. Student selects answers for objective questions (MCQ/True-False). 4. Student submits quiz (or timer expires). 5. System captures responses, compares against stored correct_answer fields, computes final score, and marks attempt complete. 6. System displays instant results and feedback to the student.
Alternative Flow	**A1: Timer Expiration (Timeout)** 4a. Time limit reached before manual submission; system automatically submits current answers.

5. UC-05: Identify At-Risk Students (Academic Analytics)

Table (4.5) Identify At-Risk Students (Academic Analytics)

Use Case ID	**UC-05**
Use Case	Identify At-Risk Students (Academic Analytics)
Description	Executes heuristic rule-based analytics to identify and flag struggling students based on unsubmitted assignments, quiz averages, and content engagement.
Actors	Teacher
Pre-Conditions	- Students must be enrolled and course activity data (submissions, quiz logs, progress) must exist.
Subsequent-Conditions	- Risk scores computed and color-coded risk flags (High/Medium/Low) rendered on teacher dashboard.
Main Flow	1. Teacher accesses "Student Performance / Analytics" dashboard for a specific course. 2. System queries student engagement metrics (missing tasks, quiz scores, completed content percentage). 3. Analytics engine calculates weighted Risk Score using the heuristic formula. 4. System classifies students into risk tiers (High > 70, Medium 40-70, Low < 40). 5. Dashboard displays list of students highlighted with corresponding visual risk badges.
Alternative Flow	**A1: Insufficient Activity Data** 2a. Course has just started; system displays "Insufficient Data for Risk Assessment" notice.

4.3 Activity Diagram:

Activity diagrams illustrate the dynamic workflow and sequential logic of complex system processes>

Diagram Legend / Key:

- **Solid Circle (Start Node):** Indicates the beginning of the workflow.
- **Circle with Border (End Node):** Indicates the completion of the workflow.
- **Rectangle (Action):** Represents a specific task or system operation.
- **Diamond (Decision Node):** Represents a conditional branch (e.g., checking time limits).
- **Arrows (Control Flow):** Indicate the sequential order of execution.

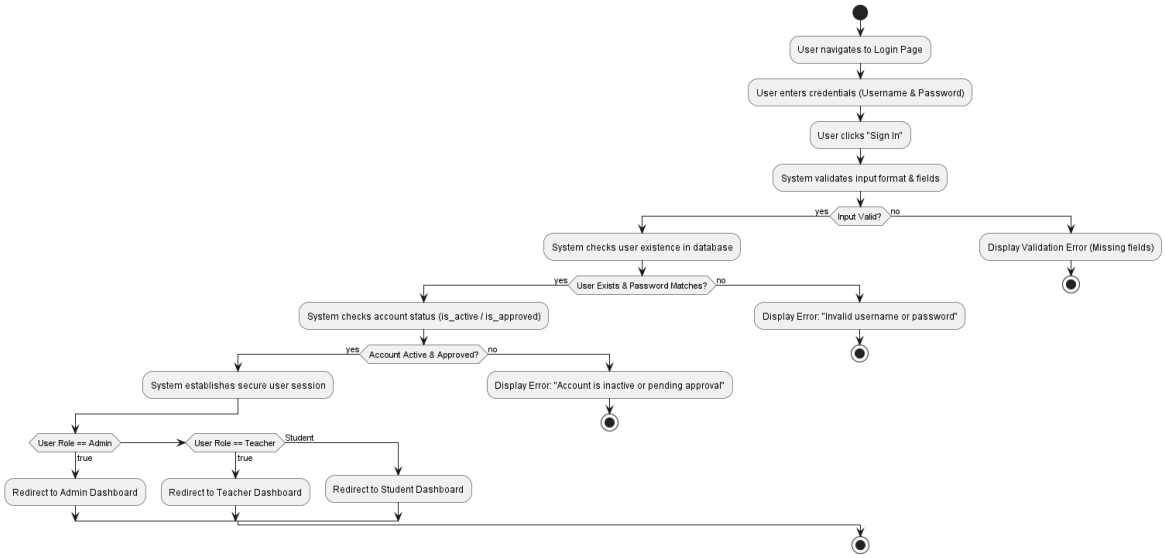


Figure (4.5) Activity Diagram for User Authentication (Sign In)

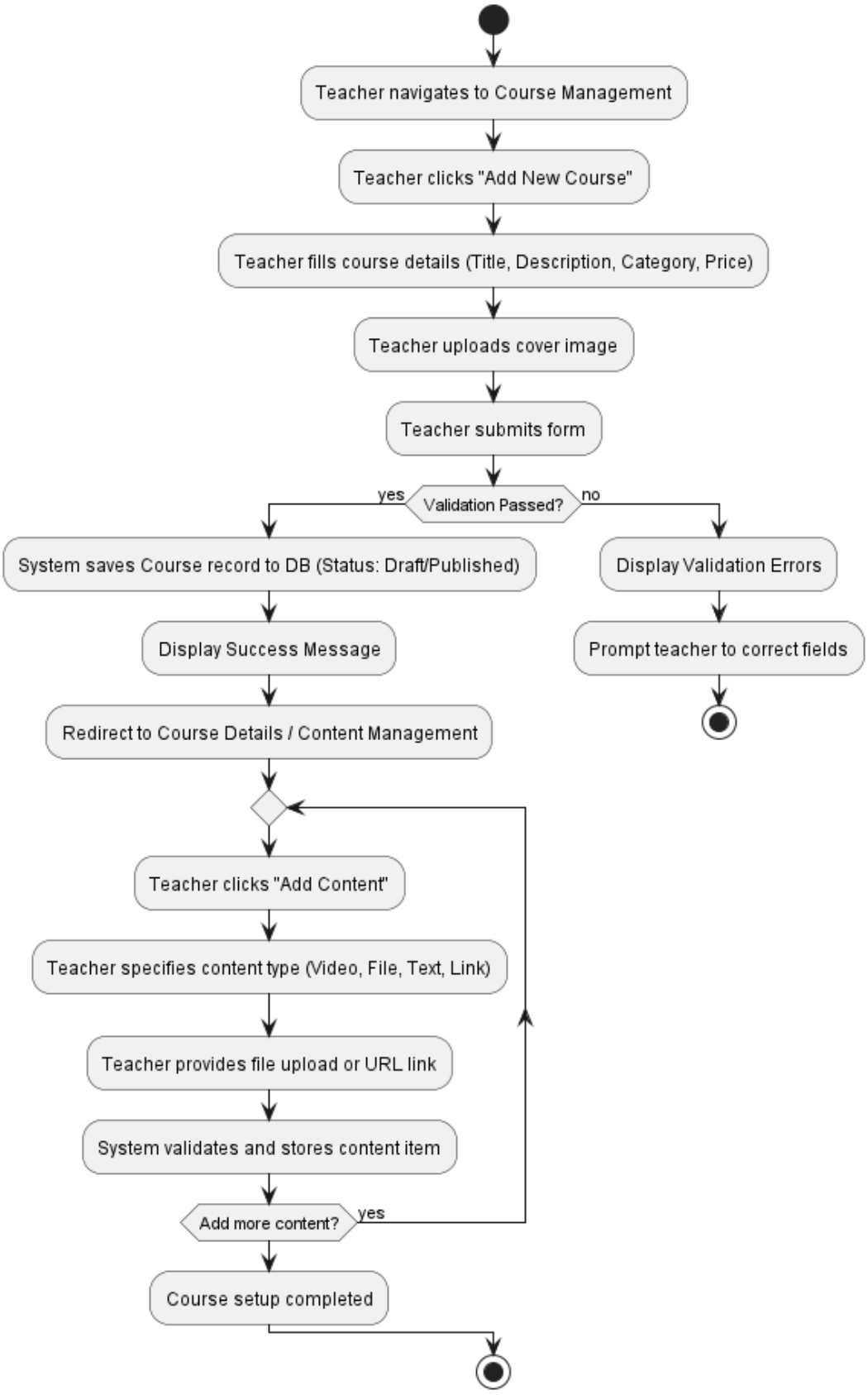


Figure (4.6) Activity Diagram for Create and Manage Course

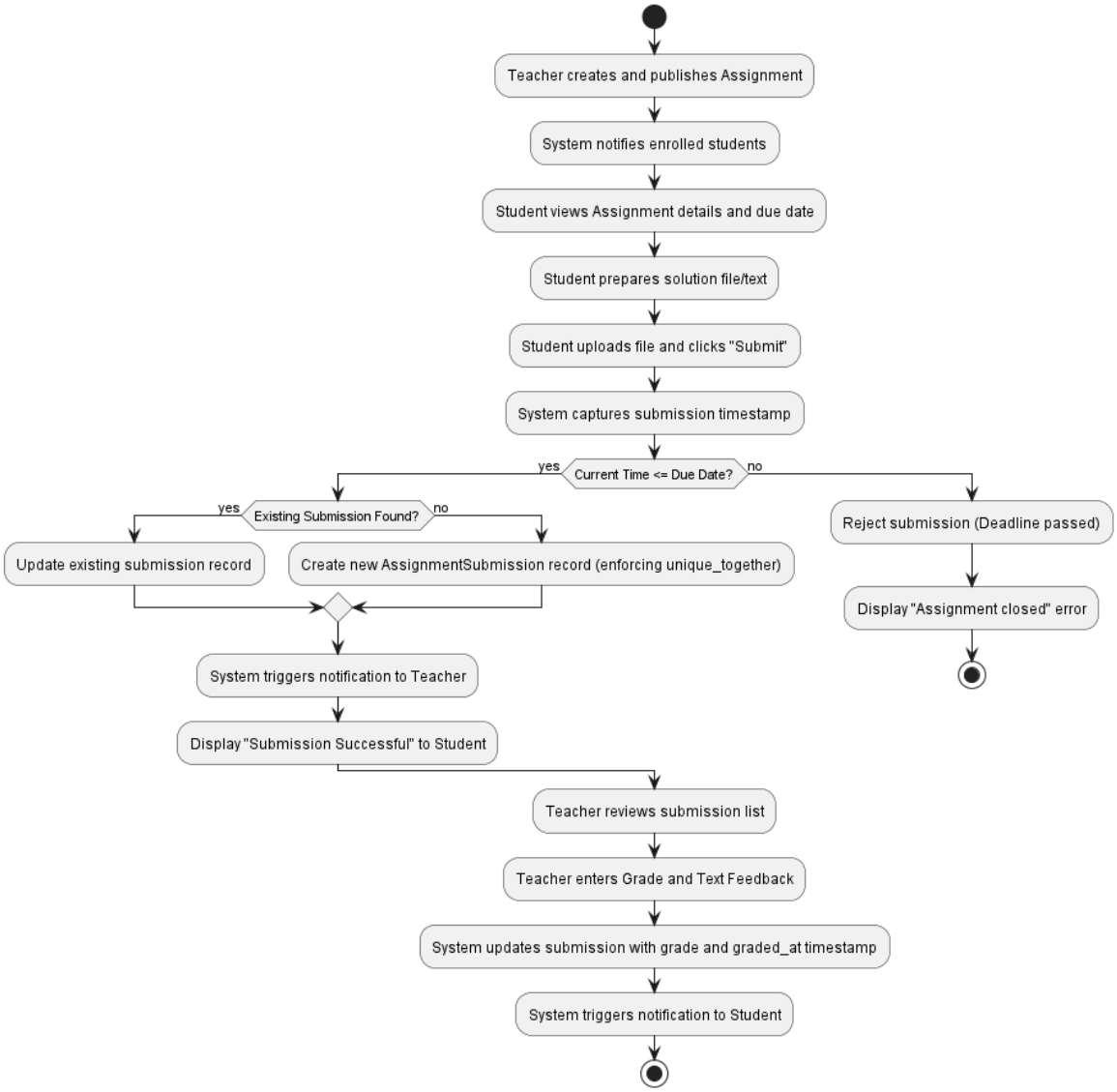


Figure (4.7) Activity Diagram Assignment Creation and Grading Workflow

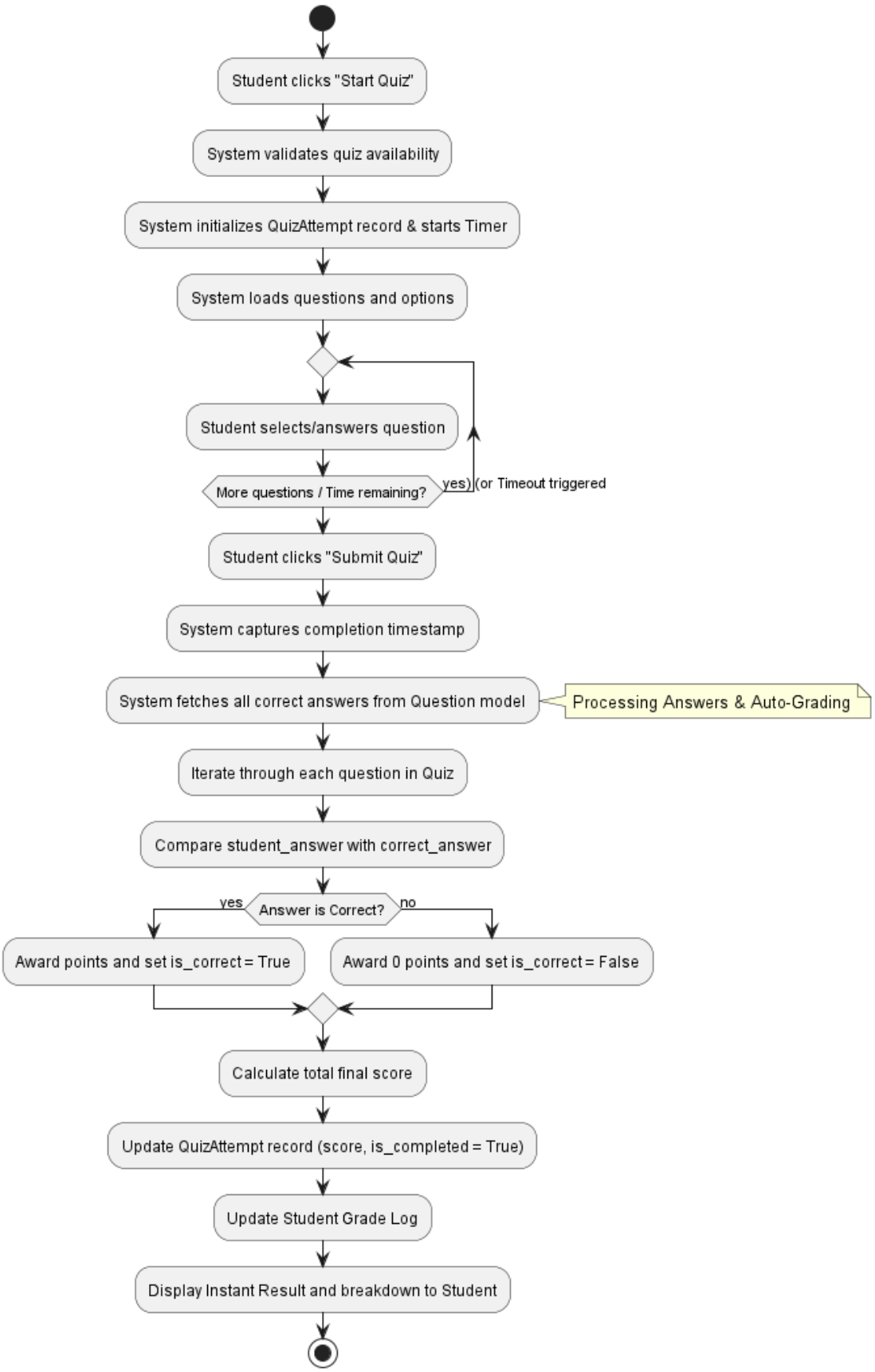


Figure (4.8) Activity Diagram Assignment Creation and Grading Workflow

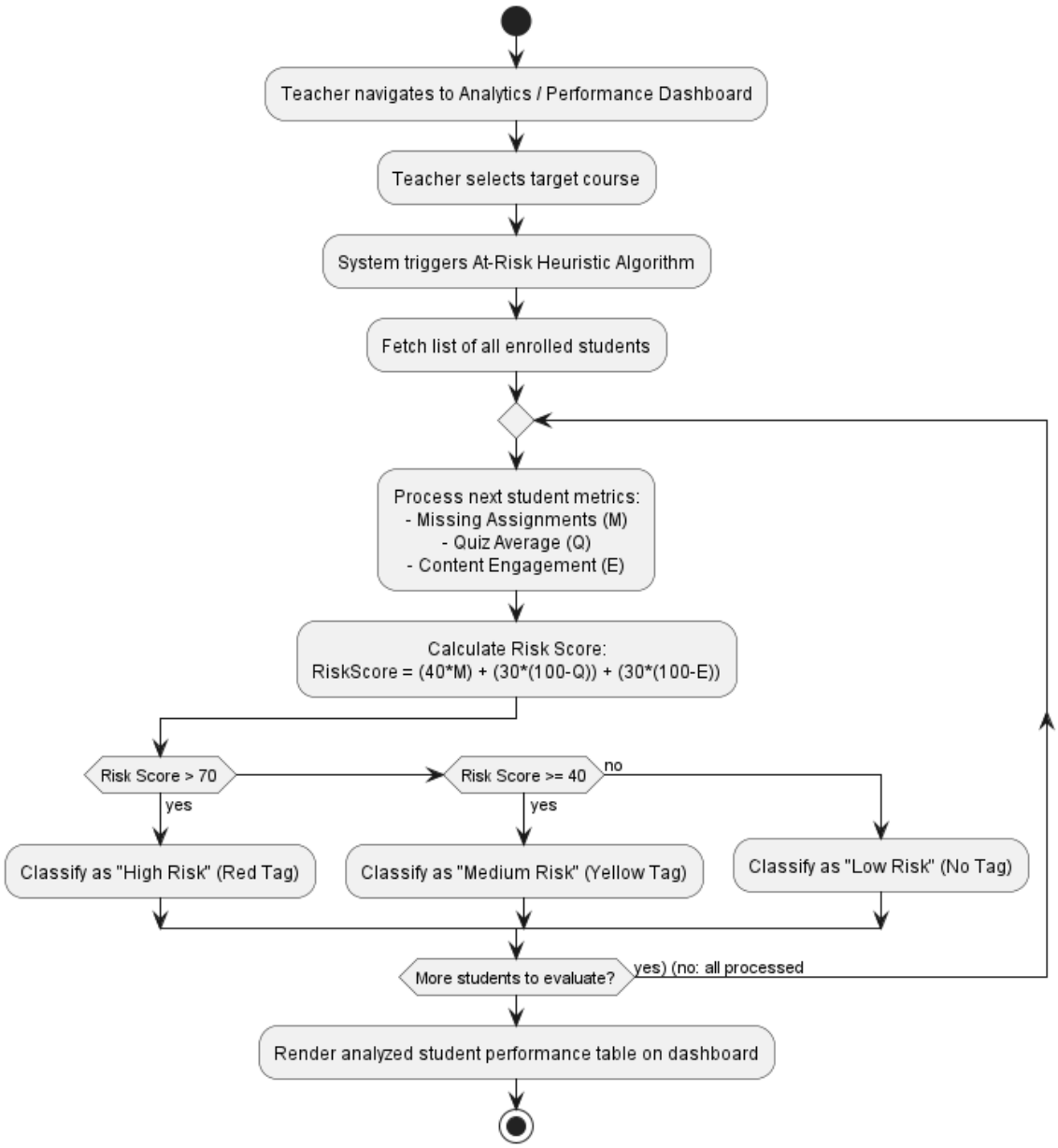


Figure (4.9) Activity Diagram for Identify At-Risk Students (Academic Analytics)

4.4 Sequence Diagram:

Sequence diagrams model the temporal exchange of messages between objects to accomplish a specific task.

Diagram Legend / Key:

- **Actor (Stick Figure):** The user initiating the action.
- **Lifelines (Dashed Lines):** Represent the existence of an object over time.
- **Messages (Solid Arrows):** Synchronous requests or method calls.
- **Return Messages (Dashed Arrows):** Responses or data returned to the caller.
- **Activation Box (Rectangle on Lifeline):** Indicates the period an object is actively processing a request.

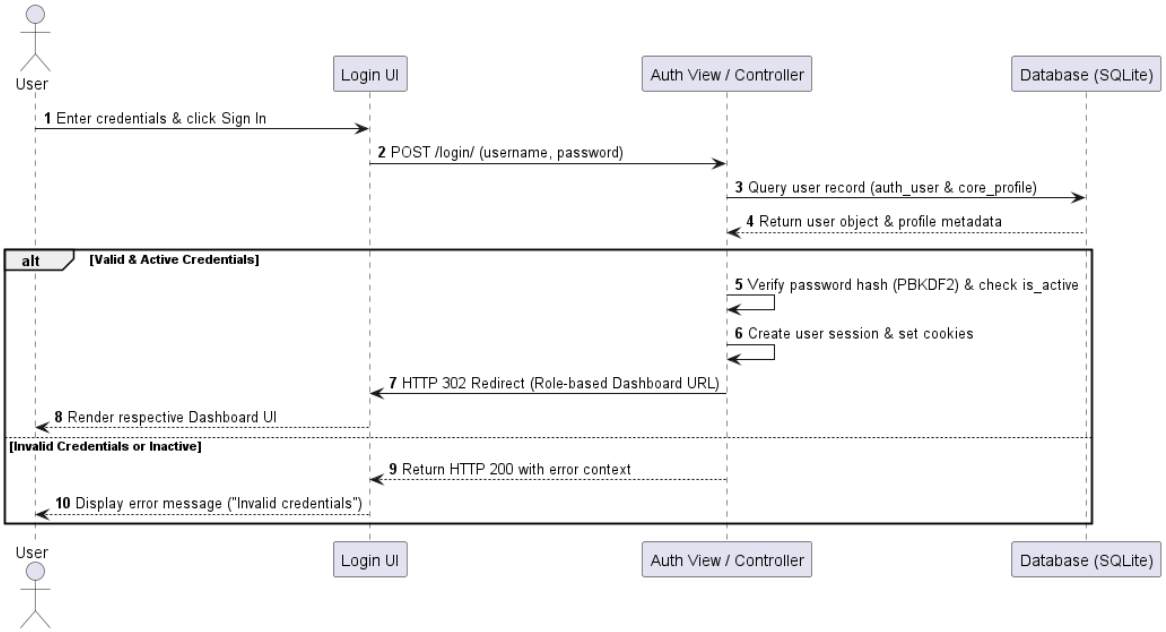


Figure (4.10) Sequence Diagram for User Authentication (Sign In)

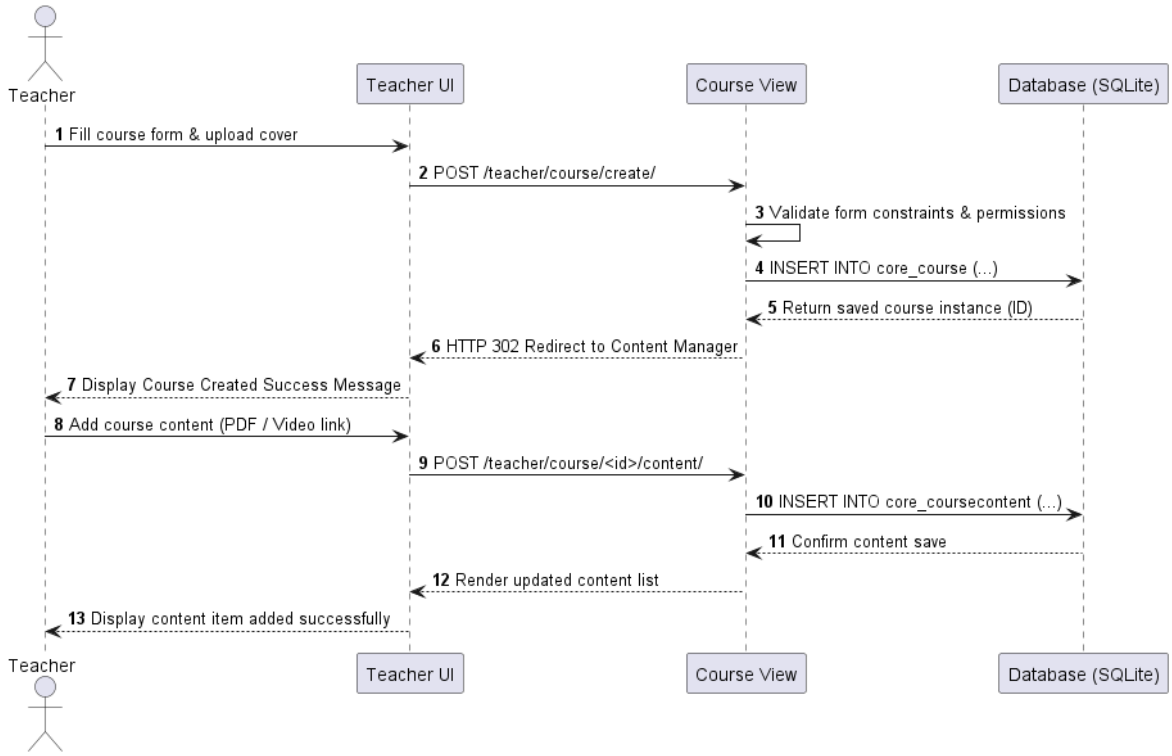


Figure (4.11) Sequence Diagram for Create and Manage Course

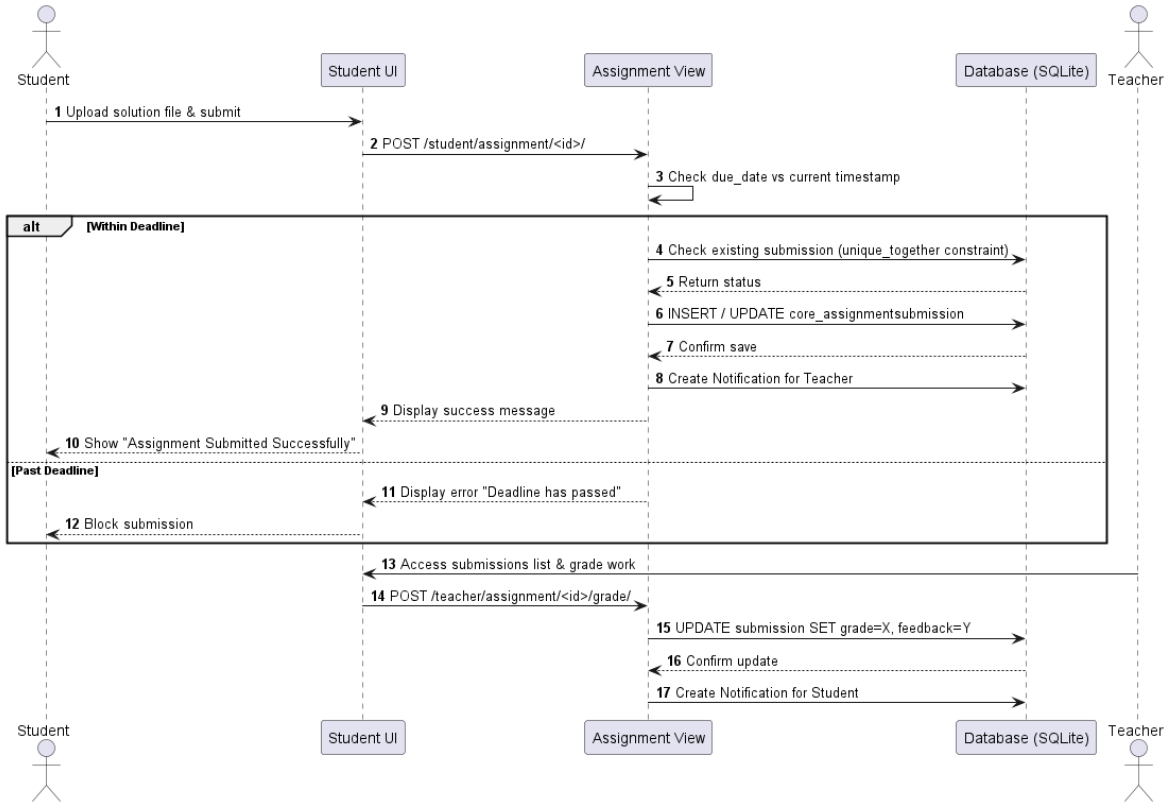


Figure (4.12) Sequence Diagram Assignment Creation and Grading Workflow

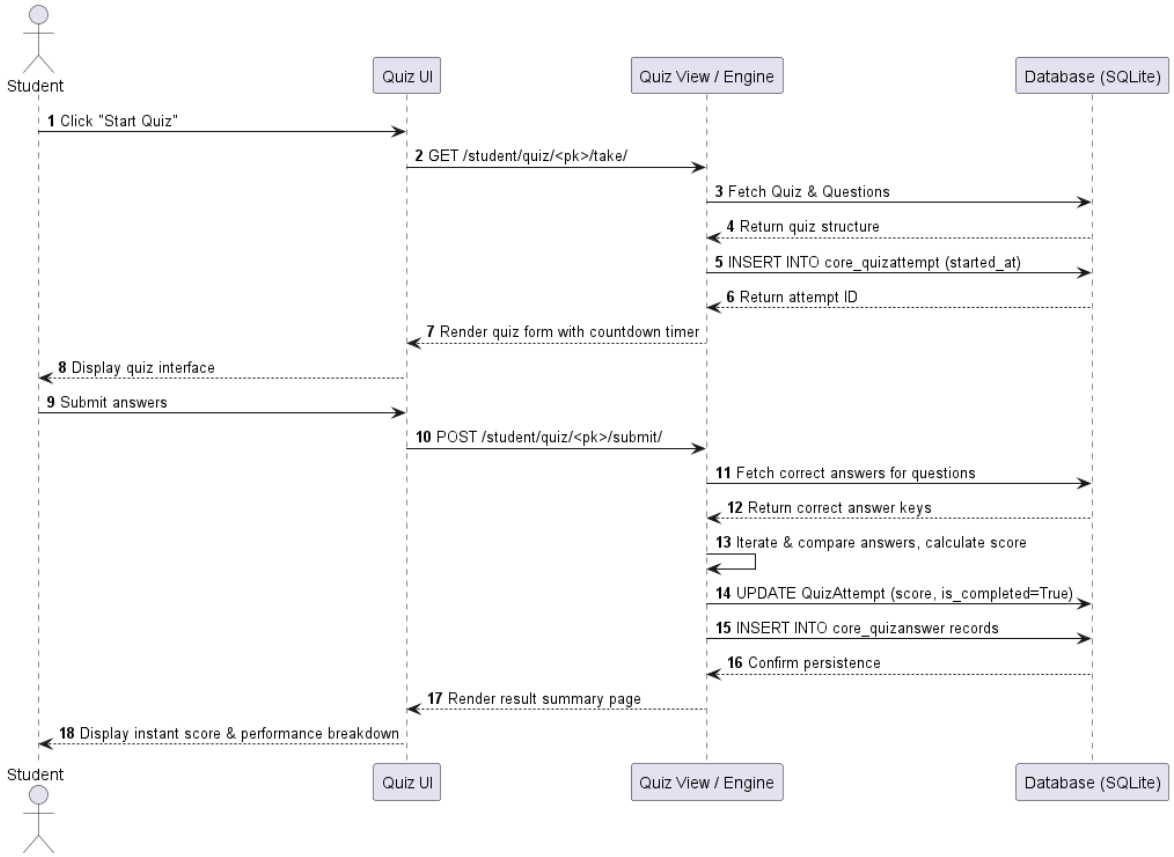


Figure (4.13) Sequence Diagram Assignment Creation and Grading Workflow

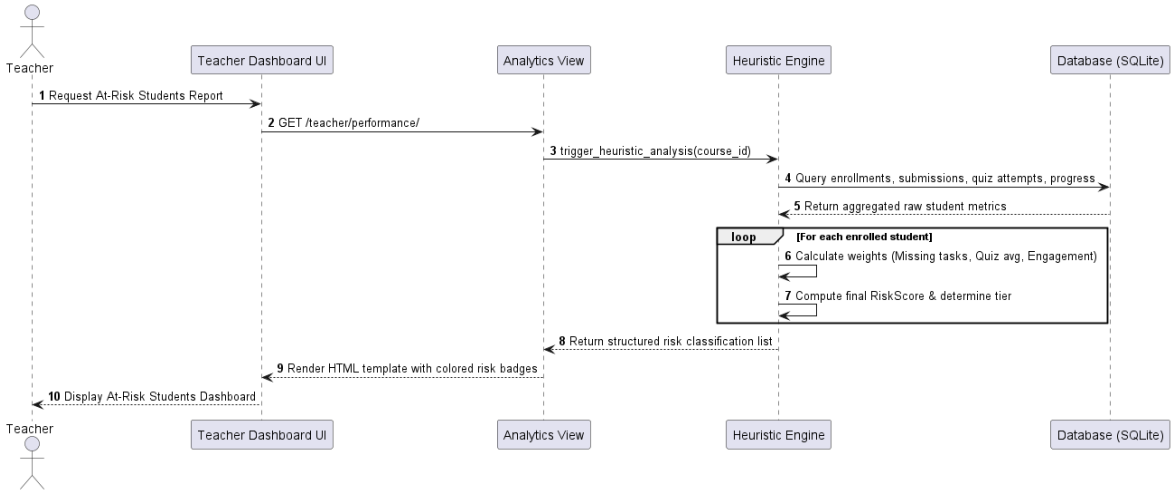


Figure (4.14) Sequence Diagram for Identify At-Risk Students (Academic Analytics)

4.5 Class Diagram:

The Class Diagram defines the static structural model of the system, mapping directly to the Django Models. It highlights the attributes, methods, and relationships (Foreign Keys, Many-to-Many) of the core entities, specifically integrating the new assessment and communication modules.

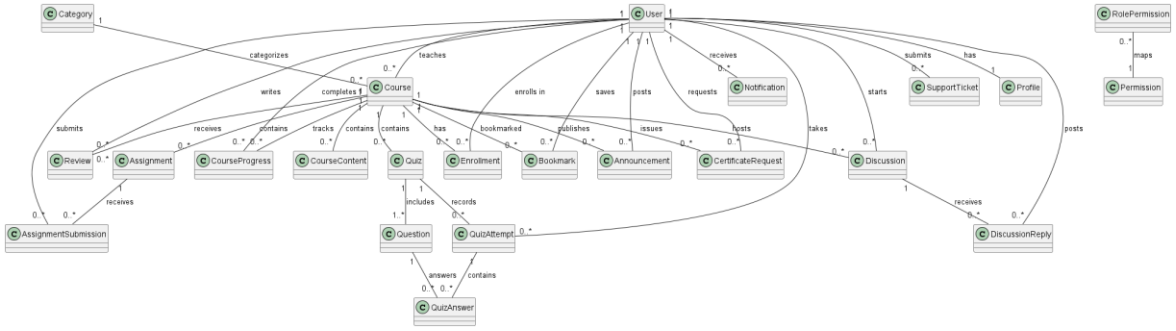


Figure (4.15) System Class Diagram

Diagram Legend / Key:

- **Class Box:** Divided into three compartments (Name, Attributes, Methods).
- **Associations (Solid Lines):** Represent structural relationships between classes.
- **Multiplicity (e.g., 1..*):** Indicates the number of instances (One-to-Many, Many-to-Many).
- **ForeignKey:** Denotes a relational database link between entities.

4.6 Component Diagram:

The Component Diagram illustrates the high-level software components and their interfaces, showing how the system is physically and logically organized for deployment.

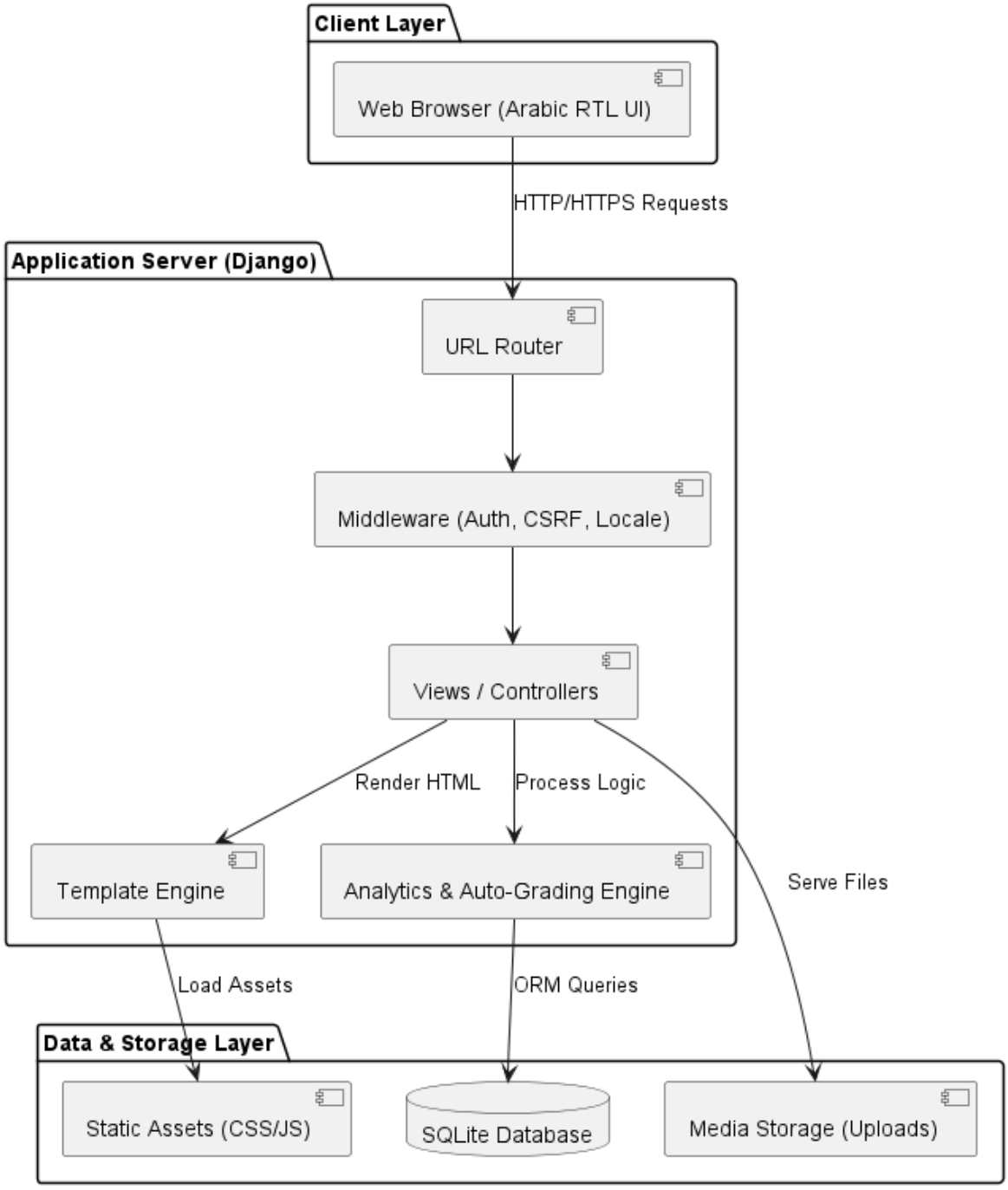


Figure (4.16) Component Diagram

Diagram Legend / Key:

- **Package (Box):** Represents a logical grouping of components.
- **Component (Box with tabs):** Represents a modular, deployable unit of the system.

- **Database (Cylinder):** Represents the persistent data storage.
- **Interfaces (Lollipop/Stick):** Represent the communication protocols (e.g., HTTP, ORM).

4.7 Entity-Relationship Diagram (ERD):

The ERD models the database schema, ensuring data integrity and normalization. The schema has been expanded to support the new functional requirements (REQ-030 to REQ-059), introducing tables for Assignments, Submissions, Quizzes, Questions, Announcements, and Discussions.

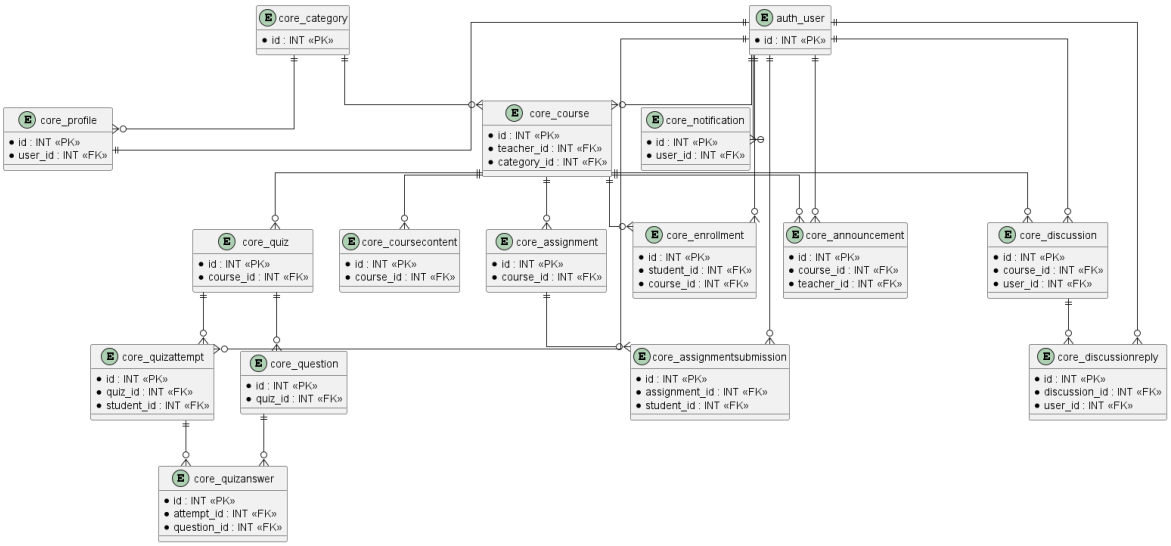


Figure (4.17) Entity-Relationship Diagram (ERD)

Table (4.6) Relational Schema Additions

Table Name	Attribute Name	Data Type	Constraints	Description
auth_user	id	INTEGER	PRIMARY KEY	Global user identifier
	username	VARCHAR(150)	NOT NULL, UNIQUE	Login username
	email	VARCHAR(254)	NOT NULL, UNIQUE	User email address
	password	VARCHAR(128)	NOT NULL	PBKDF2-SHA256 hashed password
core_profile	id	INTEGER	PRIMARY KEY	Profile record identifier
	user_id	INTEGER	FK → auth_user.id, UNIQUE	Linked user account (1-to-1)
	role	VARCHAR(20)	NOT NULL, DEFAULT 'student'	Role (admin, teacher, student)
	phone	VARCHAR(15)	NULL	Contact phone number
	bio	TEXT	NULL	User biography
	image	VARCHAR(100)	NULL	Path to profile image
	certificate	VARCHAR(100)	NULL	Path to uploaded certificate
	is_approved	BOOLEAN	NOT NULL, DEFAULT False	Account activation status

	specialty_id	INTEGER	FK → core_category.id, NULL	Teacher's specialty category
core_category	id	INTEGER	PRIMARY KEY	Unique category identifier
	name	VARCHAR(100)	NOT NULL	Category name
	description	TEXT	NULL	Category description
core_course	id	INTEGER	PRIMARY KEY	Unique course identifier
	title	VARCHAR(200)	NOT NULL	Course title
	description	TEXT	NOT NULL	Course description
	teacher_id	INTEGER	FK → auth_user.id, NOT NULL	Course instructor reference
	category_id	INTEGER	FK → core_category.id, NULL	Course category reference
	price	DECIMAL(10, 2)	NOT NULL, DEFAULT 0.00	Course fee
	status	VARCHAR(20)	NOT NULL, DEFAULT 'draft'	Status (draft, published, archived)
	created_at	DATETIME	NOT NULL	Creation timestamp
	updated_at	DATETIME	NOT NULL	Last update timestamp

core_coursecontent	id	INTEGER	PRIMARY KEY	Content item identifier
	course_id	INTEGER	FK → core_course.id, NOT NULL	Parent course reference
	title	VARCHAR(200)	NOT NULL	Content title
	content_type	VARCHAR(20)	NOT NULL	Type (video, file, text, pdf)
	link_or_file	VARCHAR(200)	NULL	External URL link
	file_upload	VARCHAR(100)	NULL	Path to uploaded file
	text_content	TEXT	NULL	Raw text content
	order	INTEGER	NOT NULL, DEFAULT 0	Sequential display order
core_quiz	id	INTEGER	PRIMARY KEY	Assessment identifier
	course_id	INTEGER	FK → core_course.id, NOT NULL	Associated course reference
	title	VARCHAR(200)	NOT NULL	Quiz title
	description	TEXT	NULL	Quiz instructions
	status	VARCHAR(20)	NOT NULL, DEFAULT 'draft'	State (draft, active, closed)
	time_limit	INTEGER	NOT NULL, DEFAULT 30	Time limit in minutes
	created_at	DATETIME	NOT NULL	Creation timestamp

core_question	id	INTEGER	PRIMARY KEY	Question identifier
	quiz_id	INTEGER	FK → core_quiz.id, NOT NULL	Parent quiz reference
	text	TEXT	NOT NULL	Question statement
	question_type	VARCHAR(20)	NOT NULL, DEFAULT 'mcq'	Type (text, mcq, true_false)
	option_a	VARCHAR(300)	NULL	MCQ Option A
	option_b	VARCHAR(300)	NULL	MCQ Option B
	option_c	VARCHAR(300)	NULL	MCQ Option C
	option_d	VARCHAR(300)	NULL	MCQ Option D
	correct_answer	VARCHAR(300)	NOT NULL	Correct answer for auto-grading
	points	INTEGER	NOT NULL, DEFAULT 1	Points awarded for correct answer
core_enrollment	id	INTEGER	PRIMARY KEY	Enrollment record identifier
	student_id	INTEGER	FK → auth_user.id, NOT NULL	Enrolled student reference
	course_id	INTEGER	FK → core_course.id, NOT NULL	Enrolled course reference

	status	VARCHAR(20)	NOT NULL, DEFAULT 'pending'	State (pending, approved, rejected)
	enrolled_at	DATETIME	NOT NULL	Enrollment timestamp
core_certificaterequest	id	INTEGER	PRIMARY KEY	Certificate request identifier
	student_id	INTEGER	FK → auth_user.id, NOT NULL	Requesting student reference
	course_id	INTEGER	FK → core_course.id, NOT NULL	Associated course reference
	status	VARCHAR(20)	NOT NULL, DEFAULT 'pending'	State (pending, issued, rejected)
	request_date	DATETIME	NOT NULL	Request submission timestamp
core_review	id	INTEGER	PRIMARY KEY	Review identifier
	student_id	INTEGER	FK → auth_user.id, NOT NULL	Reviewing student reference
	course_id	INTEGER	FK → core_course.id, NOT NULL	Reviewed course reference
	rating	INTEGER	NOT NULL, CHECK (1-5)	Star rating scale
	comment	TEXT	NOT NULL	Review comment text

	created_at	DATETIME	NOT NULL	Review timestamp
core_paymentmethod	id	INTEGER	PRIMARY KEY	Payment method identifier
	name	VARCHAR(100)	NOT NULL	Payment method name
	is_active	BOOLEAN	NOT NULL, DEFAULT True	Availability status
core_systemsettings	id	INTEGER	PRIMARY KEY	Settings record identifier
	platform_name	VARCHAR(255)	NOT NULL	Name of the LMS platform
	support_email	VARCHAR(254)	NOT NULL	Admin support email
	enable_registration	BOOLEAN	NOT NULL, DEFAULT True	Global registration toggle
	enable_certificates	BOOLEAN	NOT NULL, DEFAULT True	Global certificate toggle
core_assignment	id	INTEGER	PRIMARY KEY	Assignment identifier
	course_id	INTEGER	FK → core_course.id, NOT NULL	Associated course reference
	title	VARCHAR(200)	NOT NULL	Assignment title
	description	TEXT	NOT NULL	Assignment instructions

	due_date	DATETIME	NOT NULL	Submission deadline
	max_score	INTEGER	NOT NULL, DEFAULT 100	Maximum achievable grade
	attachment	VARCHAR(100)	NULL	Path to assignment file
	created_at	DATETIME	NOT NULL	Creation timestamp
core_assignmentsubmission	id	INTEGER	PRIMARY KEY	Submission identifier
	assignment_id	INTEGER	FK → core_assignment.id, NOT NULL	Linked assignment
	student_id	INTEGER	FK → auth_user.id, NOT NULL	Submitting student
	file_upload	VARCHAR(100)	NULL	Path to submitted file
	submitted_text	TEXT	NULL	Text-based submission
	submitted_at	DATETIME	NOT NULL	Submission timestamp
	grade	INTEGER	NULL	Assigned grade by teacher
	feedback	TEXT	NULL	Teacher's feedback text
	graded_at	DATETIME	NULL	Grading timestamp
	graded_by_id	INTEGER	FK → auth_user.id, NULL	Teacher who graded it

			UNIQUE (assignment_id, student_id)	Prevents duplicate submissions
core_quizattempt	id	INTEGER	PRIMARY KEY	Quiz attempt identifier
	quiz_id	INTEGER	FK → core_quiz.id, NOT NULL	Linked quiz
	student_id	INTEGER	FK → auth_user.id, NOT NULL	Attempting student
	started_at	DATETIME	NOT NULL	Attempt start timestamp
	completed_at	DATETIME	NULL	Attempt completion timestamp
	score	INTEGER	NOT NULL, DEFAULT 0	Achieved score
	total_points	INTEGER	NOT NULL, DEFAULT 0	Total possible points
	is_completed	BOOLEAN	NOT NULL, DEFAULT False	Completion status
core_quizanswer	id	INTEGER	PRIMARY KEY	Answer record identifier
	attempt_id	INTEGER	FK → core_quizattempt.id, NOT NULL	Linked quiz attempt
	question_id	INTEGER	FK → core_question.id, NOT NULL	Linked question
	student_answer	VARCHAR(300)	NULL	Student's provided answer

	is_correct	BOOLEAN	NOT NULL, DEFAULT False	Auto-grading result
core_announcement	id	INTEGER	PRIMARY KEY	Announcement identifier
	course_id	INTEGER	FK → core_course.id, NOT NULL	Associated course
	teacher_id	INTEGER	FK → auth_user.id, NOT NULL	Posting teacher
	title	VARCHAR(200)	NOT NULL	Announcement title
	content	TEXT	NOT NULL	Announcement body
	created_at	DATETIME	NOT NULL	Posting timestamp
core_discussion	id	INTEGER	PRIMARY KEY	Discussion thread identifier
	course_id	INTEGER	FK → core_course.id, NOT NULL	Associated course
	user_id	INTEGER	FK → auth_user.id, NOT NULL	Thread creator
	title	VARCHAR(200)	NOT NULL	Thread title
	content	TEXT	NOT NULL	Thread initial content
	created_at	DATETIME	NOT NULL	Creation timestamp
core_discussionreply	id	INTEGER	PRIMARY KEY	Reply identifier

	discussion_id	INTEGER	FK → core_discussion.id, NOT NULL	Linked discussion thread
	user_id	INTEGER	FK → auth_user.id, NOT NULL	Replying user
	content	TEXT	NOT NULL	Reply content
	created_at	DATETIME	NOT NULL	Reply timestamp
core_courseprogress	id	INTEGER	PRIMARY KEY	Progress record identifier
	student_id	INTEGER	FK → auth_user.id, NOT NULL	Student reference
	course_id	INTEGER	FK → core_course.id, NOT NULL	Course reference
	content_id	INTEGER	FK → core_coursecontent.id, NOT NULL	Completed content item
	completed_at	DATETIME	NOT NULL	Completion timestamp
			UNIQUE (student_id, content_id)	Prevents duplicate progress logs
core_bookmark	id	INTEGER	PRIMARY KEY	Bookmark identifier
	student_id	INTEGER	FK → auth_user.id, NOT NULL	Student reference
	course_id	INTEGER	FK → core_course.id, NOT NULL	Bookmarked course
	created_at	DATETIME	NOT NULL	Bookmark timestamp

			UNIQUE (student_id, course_id)	Prevents duplicate bookmarks
core_supportticket	id	INTEGER	PRIMARY KEY	Ticket identifier
	student_id	INTEGER	FK → auth_user.id, NOT NULL	Submitting user
	subject	VARCHAR(200)	NOT NULL	Ticket subject
	message	TEXT	NOT NULL	Ticket description
	status	VARCHAR(20)	NOT NULL, DEFAULT 'open'	State (open, replied, closed)
	response	TEXT	NULL	Admin's response text
	created_at	DATETIME	NOT NULL	Creation timestamp
	updated_at	DATETIME	NOT NULL	Last update timestamp
core_notification	id	INTEGER	PRIMARY KEY	Notification identifier
	user_id	INTEGER	FK → auth_user.id, NOT NULL	Target user
	title	VARCHAR(200)	NOT NULL	Notification title
	message	TEXT	NOT NULL	Notification body
	url	VARCHAR(500)	NULL	Deep link URL
	is_read	BOOLEAN	NOT NULL, DEFAULT False	Read status

	created_at	DATETIME	NOT NULL	Creation timestamp
core_academicevent	id	INTEGER	PRIMARY KEY	Event identifier
	user_id	INTEGER	FK → auth_user.id, NOT NULL	Target user (for calendar)
	title	VARCHAR(200)	NOT NULL	Event title
	description	TEXT	NULL	Event details
	event_date	DATETIME	NOT NULL	Scheduled date/time
	event_type	VARCHAR(20)	NOT NULL, DEFAULT 'other'	Type (exam, assignment, etc.)
	course_id	INTEGER	FK → core_course.id, NULL	Associated course (if any)
core_coursehistory	id	INTEGER	PRIMARY KEY	History log identifier
	course_id	INTEGER	FK → core_course.id, NOT NULL	Modified course
	changed_by_id	INTEGER	FK → auth_user.id, NOT NULL	User who made the change
	field_changed	VARCHAR(100)	NOT NULL	Name of the modified field
	old_value	TEXT	NULL	Previous field value
	new_value	TEXT	NULL	New field value

	changed_at	DATETIME	NOT NULL	Modification timestamp
core_permission	id	INTEGER	PRIMARY KEY	Permission identifier
	name	VARCHAR(100)	NOT NULL, UNIQUE	Human-readable permission name
	codename	VARCHAR(100)	NOT NULL, UNIQUE	Machine-readable codename
	description	TEXT	NULL	Permission description
core_rolepermission	id	INTEGER	PRIMARY KEY	Role-Permission mapping ID
	role	VARCHAR(20)	NOT NULL	User role (admin, teacher, etc.)
	permission_id	INTEGER	FK → core_permission.id, NOT NULL	Linked permission
			UNIQUE (role, permission_id)	Prevents duplicate role mappings

4.8 Sitemap:

The Sitemap defines the hierarchical navigation structure of the Arabic RTL interface, ensuring intuitive access to the newly implemented modules.

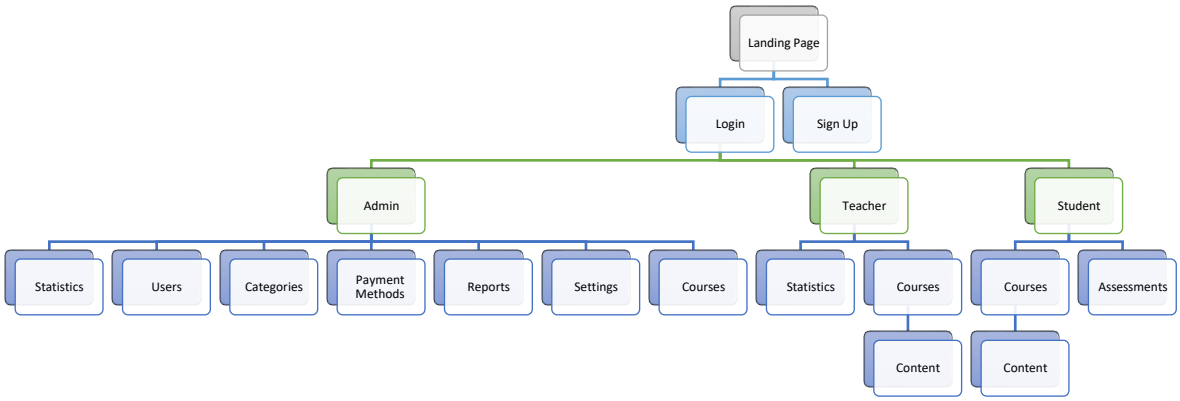


Figure (4.18) System Sitemap

Navigation Hierarchy:

1. **Public Pages:** Landing Page, Login, Registration.
2. **Student Dashboard:**
    - My Courses →→ Course Content, Assignments, Quizzes, Discussions.
    - Announcements & Notifications.
    - Academic Calendar & Progress Tracker.
3. **Teacher Dashboard:**
    - Course Management →→ Create/Edit Course, Upload Content.
    - Assessment Center →→ Create Assignment, Design Quiz, Grade Submissions.
    - Communication →→ Post Announcement, Manage Discussions.

- Analytics →→ Student Progress, Identify At-Risk Students.

4. **Admin Panel:**
    - User Management, Categories, System Settings, Global Statistics, Database Backup.

4.9 Initial Test Cases (Unit Tests):

To ensure the reliability of the newly implemented modules (REQ-030 to REQ-059), specific unit test cases were designed using Django's TestCase framework.

Table (4.7) Initial Unit Test Cases

ID	Target Req	Description	Pre-conditions	Test Steps	Expected Result
UT-030	REQ-030	Verify Assignment model creation	Teacher exists	1. Instantiate Assignment object. 2. Call .save()	Object saved to DB; due_date validated.
UT-037	REQ-037	Verify auto-grading logic	Quiz & Questions exist	1. Pass correct/incorrect answers to grading function. 2. Calculate score.	Function returns exact score based on correct_answer field.
UT-048	REQ-048	Verify At-Risk heuristic logic	Student progress data exists	1. Input student metrics into heuristic function. 2. Evaluate threshold.	Function returns Risk Score and correct classification (High/Med/Low).

4.10 AI and Algorithms Design:

While the system does not utilize heavy machine learning models in this academic phase, it implements a Rule-Based Heuristic Algorithm to fulfill REQ-048 (Identify At-Risk Students). This algorithm transitions the LMS from a passive tool to a proactive support system.

Algorithm Logic: At-Risk Student Identification The algorithm calculates a Risk Score for each student enrolled in a course based on three weighted parameters:

1. **Missing Assignments (**M*M*)**:** Number of assignments past due without submission.
2. **Quiz Average (**Q*Q*)**:** Average grade across all taken quizzes (normalized to 0-100).
3. **Content Engagement (**E*E*)**:** Percentage of course content viewed.

Heuristic Formula:

RiskScore=(W1×M)+(W2×(100−Q))+(W3×(100−E))*RiskScore*=(*W*1 ×*M*)+(*W*2×(100−*Q*))+(*W*3×(100−*E*)) *Where* W1,W2,W3*W*1,*W*2,*W*3 *are configurable weights (Default:* W1=40,W2=30,W3=30*W*1=40,*W*2=30,*W*3 =30*).*

Threshold Classification:

- **Risk Score > 70:** High Risk (Flagged in Red on Teacher Dashboard).
- **Risk Score 40 - 70:** Medium Risk (Flagged in Yellow).
- **Risk Score < 40:** Low Risk (No flag).

This deterministic approach ensures transparency, requires minimal computational overhead, and provides teachers with actionable, interpretable data without the "black box" nature of complex neural networks.

4.11 Requirements Traceability Matrix (RTM):

The RTM ensures bidirectional traceability between the 59 functional requirements, the design artifacts (Use Cases, Class Diagrams), and the test

cases. Below is the updated RTM, explicitly integrating the newly implemented modules (REQ-037 to REQ-048).

Table (4.8) Requirements Traceability Matrix (RTM)

REQ-030	Create Assignment	UC-03	Assignment Model	UT-030
REQ-037	Auto-Grade Objective Qs	UC-04	QuizAnswer, Question	UT-037
REQ-048	Identify At-Risk Students	UC-07	CourseProgress, Submission	UT-048

Chapter 5
Implementation

5.1 Implementation Environment:

The system was developed and tested within a controlled, reproducible academic environment utilizing a curated suite of open-source, industry-standard tools. The selection of these tools was driven by zero-licensing costs, cross-platform compatibility, and alignment with IEEE software engineering best practices.

- **Operating System:** Windows 11 (Development and Testing Environment).
- **Programming Language:** Python 3.10+ (Selected for its readability, robust standard library, and mature web development ecosystem).
- **Backend Framework:** Django 4.2 LTS (Long Term Support), utilizing the Model-View-Template (MVT) architectural pattern.
- **Database Engine:** SQLite 3 (Embedded, serverless RDBMS used for academic prototyping, managed via Django ORM).
- **Frontend Technologies:** HTML5, CSS3, JavaScript, and Bootstrap 5.3 (Official RTL Distribution for native Arabic interface rendering).
- **Integrated Development Environment (IDE):** Visual Studio Code (VS Code) with Python, Pylance, and Django extensions.
- **Version Control:** Git (Local repository management) and GitHub (Remote hosting and collaboration).
- **Database Management:** DB Browser for SQLite (Schema visualization and manual query validation).